Rapport Projet Architecture web Distribuée

Une plateforme numérique qui facilite la découverte des restaurants, la consultation des menus et la commande de repas à emporter ou en livraison.

Elaborée par :

Douaa Ben Rejab

Maryem bdira filali

Manel hosni

Mahdi ben Fattoum

Année universitaire :2026/2027

PARTIE I : PRÉSENTATION GÉNÉRALE DU PROJET

Introduction

Dans le domaine de la restauration, les nouvelles technologies jouent un rôle important dans l'amélioration des services proposés aux clients et dans la gestion quotidienne des restaurants. Aujourd'hui, les clients recherchent des solutions simples et rapides pour découvrir les restaurants, consulter leurs menus et commander des repas sans avoir à se déplacer.

Dans ce contexte, notre projet consiste à concevoir et développer **YallaFood**, une plateforme numérique dédiée à la découverte des restaurants, à la commande de repas et à la livraison à domicile.

Cette plateforme vise à regrouper plusieurs restaurants au sein d'une seule application, tout en facilitant la gestion des commandes, des menus et des livraisons pour les différents acteurs.

1.1. Présentation générale du projet

YallaFood est une plateforme numérique qui met en relation les clients et les restaurateurs à travers une interface simple et intuitive.

Elle permet aux clients de consulter les restaurants disponibles, de découvrir leurs menus et leurs prix, de commander des plats à emporter ou de demander une livraison à domicile.

La plateforme offre également aux restaurateurs la possibilité de gérer les informations de leur restaurant, leurs menus, leurs plats et les commandes reçues. Les livreurs peuvent consulter les livraisons qui leur sont attribuées et mettre à jour leur état d'avancement.

Le projet repose sur une architecture microservices afin de diviser l'application en plusieurs services indépendants. Chaque service assure une responsabilité spécifique et communique avec les autres à travers des interfaces de programmation (API).

1.2. Problématique

La gestion des restaurants et des commandes peut présenter plusieurs difficultés.

Les clients doivent parfois utiliser différents moyens pour consulter les menus, connaître les prix, contacter les restaurants ou commander des repas. De leur côté, les restaurateurs peuvent rencontrer des difficultés dans la gestion simultanée des menus, des disponibilités et des commandes.

La gestion des livraisons représente également un défi, notamment pour l'affectation des livreurs et le suivi de l'état des commandes.

Face à ces difficultés, la question principale qui se pose est la suivante :

Comment concevoir une plateforme numérique centralisée permettant de simplifier la gestion des restaurants, de faciliter la commande de repas et d'assurer un suivi efficace des livraisons ?

1.3. Objectifs du projet

L'objectif principal de YallaFood est de développer une solution numérique complète permettant d'améliorer l'expérience des clients et de simplifier le travail des restaurateurs.

Les objectifs spécifiques sont :

- Centraliser plusieurs restaurants au sein d'une seule plateforme.
- Faciliter la consultation des restaurants et de leurs menus.
- Permettre aux clients de consulter les descriptions et les prix des plats.
- Offrir la possibilité de commander des repas à emporter ou en livraison.
- Faciliter la gestion et le suivi des commandes.
- Permettre aux restaurateurs de gérer leurs menus, leurs plats et leurs disponibilités.
- Faciliter l'affectation des livreurs et le suivi des livraisons.
- Assurer une communication efficace entre les différents services de l'application.
- Garantir la sécurité des données et des utilisateurs.

1.4. Solution proposée

Pour répondre à la problématique identifiée, nous proposons le développement de la plateforme YallaFood.

La solution repose sur une architecture distribuée basée sur les microservices. Cette approche permet de diviser l'application en plusieurs composants indépendants, chacun responsable d'une fonctionnalité précise.

La plateforme est composée de cinq microservices principaux :

- **Microservice Utilisateur :** assure la gestion des utilisateurs, l'authentification et le contrôle des droits d'accès.
- **Microservice Restaurant :** gère les informations des restaurants, leurs horaires et leurs disponibilités.
- **Microservice Menu :** assure la gestion des menus, des plats, des prix et de leur disponibilité.
- **Microservice Commande :** gère la création, la consultation, la confirmation, le suivi et l'annulation des commandes.
- **Microservice Livraison :** assure la gestion des livreurs, l'affectation des livraisons et le suivi de leur avancement.

Chaque microservice dispose de sa propre base de données afin de garantir son indépendance.

Les différents services communiquent principalement à travers des API REST pour les échanges synchrones et RabbitMQ pour les communications asynchrones.

Cette architecture facilite la maintenance, l'évolution et l'intégration de nouvelles fonctionnalités.

2. Technologies utilisées

Les technologies retenues pour le développement de la plateforme YallaFood sont les suivantes :

- **Architecture :** Microservices, pour découper la plateforme en services indépendants.
- **Framework :** Spring Boot (Spring Web, Spring Data JPA), pour développer les microservices.

- **Bases de données :** MySQL, chaque microservice disposant de sa propre base de données.
- **Communication synchrone :** API REST (protocole HTTP, format JSON) et OpenFeign, pour les échanges immédiats entre les services.
- **Communication asynchrone :** RabbitMQ, pour l'échange d'événements entre les services.
- **Point d'entrée :** API Gateway, pour le routage des requêtes vers les microservices.
- **Découverte des services :** Eureka, pour l'enregistrement et la localisation des microservices.
- **Sécurité :** JWT, pour l'authentification et le contrôle des rôles.
- **Documentation des API :** Swagger / OpenAPI, pour décrire et tester les endpoints.

3. Les acteurs de la plateforme

Acteurs principaux

Client :

- Consulte les restaurants disponibles.
- Découvre les menus et les prix.
- Commande des repas à emporter ou en livraison.
- Consulte et suit ses commandes.
- Reçoit les informations relatives à ses commandes et livraisons.

Restaurateur :

- Gère les informations de son restaurant.
- Gère ses horaires et ses disponibilités.
- Ajoute, modifie et supprime les plats de son menu.
- Confirme ou refuse les commandes reçues.
- Suit la préparation des commandes.

Administrateur :

- Valide ou refuse l'ajout des restaurants.
- Supervise les restaurants et les menus.
- Gère les livreurs.

- Assigne ou annule les livraisons.

Livreur :

- Consulte les livraisons qui lui sont assignées.
- Met à jour le statut des livraisons.
- Envoie sa position.
- Gère sa disponibilité.

Acteur secondaire (système)

Microservice Utilisateur :

Assure l'authentification des utilisateurs, vérifie leur identité et contrôle leurs droits d'accès selon leurs rôles (client, restaurateur, administrateur et livreur).

4. Architecture globale de la plateforme

La plateforme YallaFood repose sur une architecture microservices composée de cinq services indépendants :

1. Microservice Utilisateur.
2. Microservice Restaurant.
3. Microservice Menu.
4. Microservice Commande.
5. Microservice Livraison.

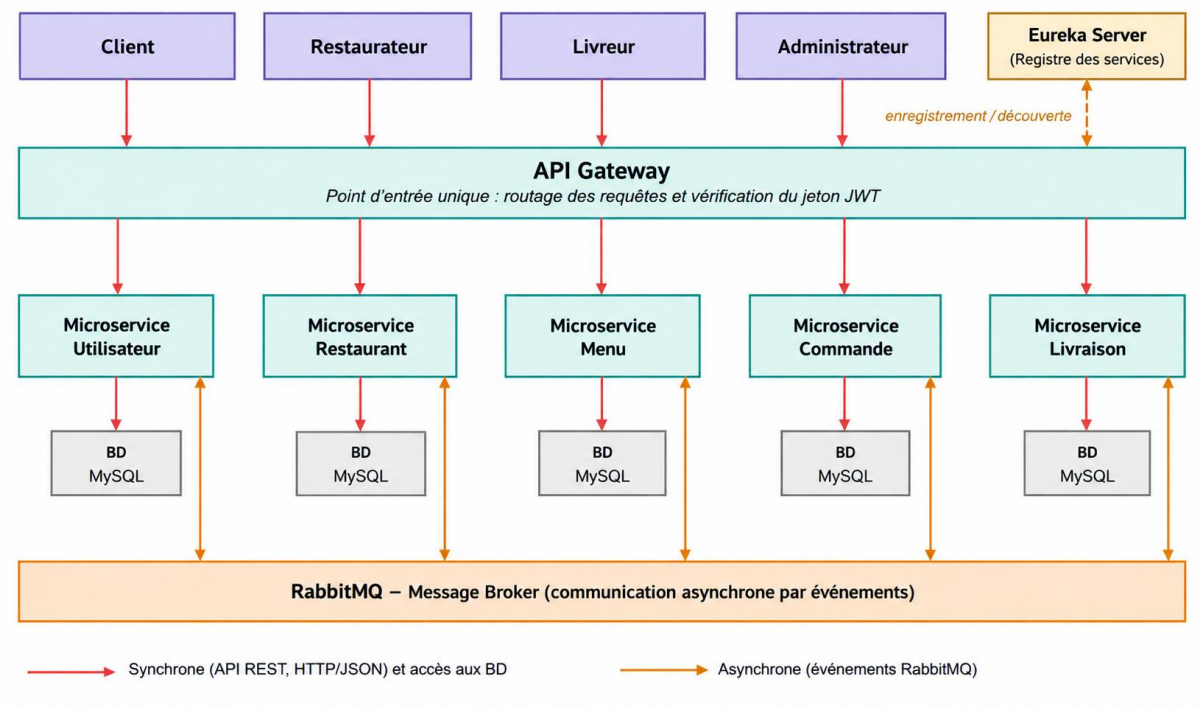
Les utilisateurs (client, restaurateur, administrateur et livreur) accèdent à l'application à travers l'API Gateway. Elle constitue le point d'entrée unique de la plateforme : elle route chaque requête vers le microservice concerné et participe à la sécurisation des accès grâce à la vérification des jetons JWT.

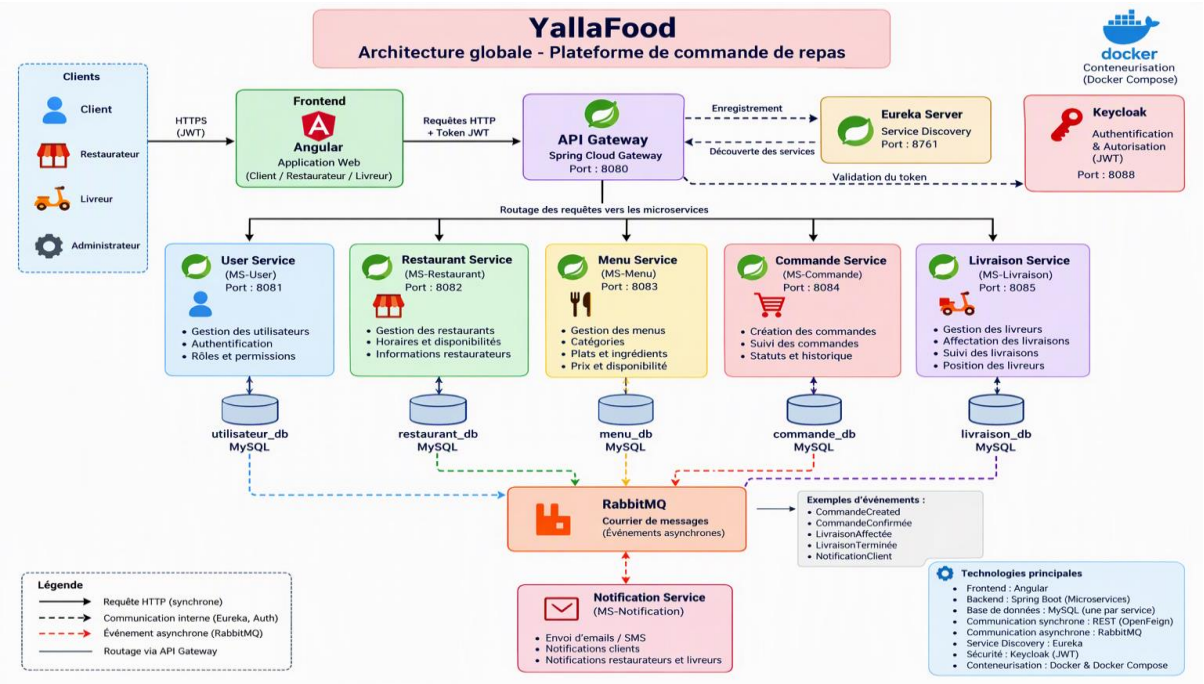
Les microservices s'enregistrent auprès du serveur Eureka, ce qui permet de les localiser et de faciliter leur communication entre eux.

Chaque microservice dispose de sa propre base de données, ce qui garantit son indépendance et l'isolation de ses données.

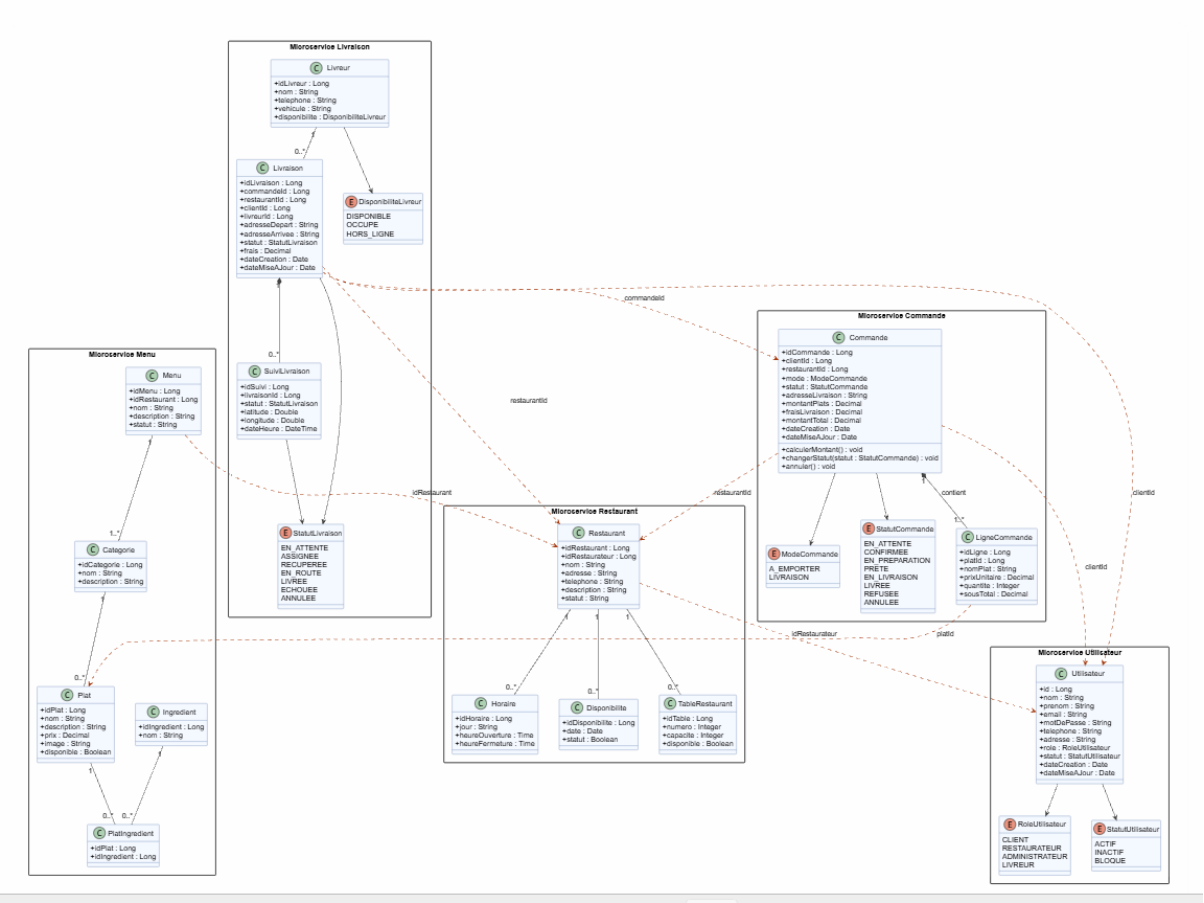
Les échanges immédiats entre les services sont réalisés à travers les API REST et OpenFeign, tandis que les changements importants sont diffusés sous forme d'événements par l'intermédiaire de RabbitMQ.

Cette architecture permet d'obtenir une plateforme modulaire, évolutive et facile à maintenir.





5. Diagramme de classes globale


Microservice Livraison
Livreur
+idLivreur : Long
+nom : String
+telephone : String
+vehicule : String
+disponibilite : DisponibiliteLivreur
Livraison
+idLivraison : Long
+commandeId : Long
+restaurantId : Long
+clientId : Long
+livreurId : Long
+adresseDepart : String
+adresseArrivee : String
+statut : StatutLivraison
+frais : Decimal
+dateCreation : Date
+dateMiseAJour : Date
DisponibiliteLivreur
DISPONIBLE
OCCUPE
HORS_LIGNE
SuiviLivraison
+idSuivi : Long
+livraisonId : Long
+statut : StatutLivraison
+latitude : Double
+longitude : Double
+dateHeure : DateTime
StatutLivraison
EN_ATTENTE
ASSIGNEE
RECUPEREE
EN_ROUTE
LIVREE
ECHOUEE
ANNULEE
Microservice Menu
Menu
+idMenu : Long
+idRestaurant : Long
+nom : String
+description : String
+statut : String
Categorie
+idCategorie : Long
+nom : String
+description : String
Plat
+idPlat : Long
+nom : String
+description : String
+prix : Decimal
+image : String
+disponible : Boolean
Ingredient
+idIngredient : Long
+nom : String
PlatIngredient
+idPlat : Long
+idIngredient : Long
Microservice Restaurant
Restaurant
+idRestaurant : Long
+idRestaurateur : Long
+nom : String
+adresse : String
+telephone : String
+description : String
+statut : String
Horaire
+idHoraire : Long
+jour : String
+heureOuverture : Time
+heureFermeture : Time
Disponibilite
+idDisponibilite : Long
+date : Date
+statut : Boolean
TableRestaurant
+idTable : Long
+numero : Integer
+capacite : Integer
+disponible : Boolean
Microservice Commande
Commande
+idCommande : Long
+clientId : Long
+restaurantId : Long
+mode : ModeCommande
+statut : StatutCommande
+adresseLivraison : String
+montantPlats : Decimal
+fraisLivraison : Decimal
+montantTotal : Decimal
+dateCreation : Date
+dateMiseAJour : Date
+calculerMontant() : void
+changerStatut(statut : StatutCommande) : void
+annuler() : void
ModeCommande
A_EMPORTER
LIVRAISON
StatutCommande
EN_ATTENTE
CONFIRMEE
EN_PREPARATION
PRETE
EN_LIVRAISON
LIVREE
REFUSEE
ANNULEE
LigneCommande
+idLigne : Long
+platId : Long
+nomPlat : String
+prixUnitaire : Decimal
+quantite : Integer
+sousTotal : Decimal
Microservice Utilisateur
Utilisateur
+id : Long
+nom : String
+prenom : String
+email : String
+motDePasse : String
+telephone : String
+adresse : String
+role : RoleUtilisateur
+statut : StatutUtilisateur
+dateCreation : Date
+dateMiseAJour : Date
RoleUtilisateur
CLIENT
RESTAURATEUR
ADMINISTRATEUR
LIVREUR
StatutUtilisateur
ACTIF
INACTIF
BLOQUE
commandeId
restaurantId
idRestaurant
clientId
contient
platId
idRestaurateur

Microservice Utilisateur

1. Présentation du microservice Utilisateur

Le microservice Utilisateur est un composant indépendant de l'architecture microservices de la plateforme YallaFood. Il est responsable de la gestion des comptes et des informations personnelles des utilisateurs.

Son objectif principal est de permettre aux différents utilisateurs de créer un compte, de se connecter à la plateforme et de gérer leurs informations personnelles selon leur rôle.

Ses principales responsabilités sont :

- Créer un nouveau compte utilisateur.
- Gérer l'authentification et la connexion.
- Vérifier les informations de connexion.
- Gérer les informations personnelles des utilisateurs.
- Modifier les informations du profil.
- Gérer les rôles des utilisateurs (Client, Restaurateur, Administrateur, Livreur).
- Activer ou désactiver un compte.
- Consulter les informations d'un utilisateur.
- Gérer les mots de passe.
- Générer et valider les informations nécessaires à l'authentification JWT.

Technologies utilisées :

- Architecture : Microservices.
- Communication : API REST.
- Base de données : MySQL.
- Format d'échange : JSON.

- Protocole : HTTP.
- Authentification : JWT.
- Communication asynchrone : RabbitMQ.

2. Acteurs et fonctionnalités

Le microservice Utilisateur interagit avec différents acteurs afin d'assurer la gestion des comptes de la plateforme.

1. Le Client

Le client est un utilisateur de la plateforme. Il peut :

- Créer un nouveau compte.
- Se connecter à son compte.
- Consulter son profil.
- Modifier ses informations personnelles.
- Modifier son mot de passe.
- Consulter ses informations de compte.

2. Le Restaurateur

Le restaurateur utilise la plateforme pour gérer son espace professionnel. Il peut :

- Créer un compte restaurateur.
- Se connecter à son espace professionnel.
- Consulter ses informations personnelles.
- Modifier ses informations de compte.
- Gérer son profil.

3. Le Livreur

Le livreur utilise la plateforme pour accéder à son espace de livraison. Il peut :

- Créer un compte livreur.
- Se connecter à son espace.
- Consulter son profil.
- Modifier ses informations personnelles.

4. L'Administrateur

L'administrateur assure la gestion globale des utilisateurs. Il peut :

- Consulter la liste des utilisateurs.
- Rechercher un utilisateur.
- Consulter les détails d'un compte.
- Modifier les informations d'un utilisateur.
- Activer ou désactiver un compte.
- Gérer les rôles des utilisateurs.
- Supprimer un compte selon les règles du système.

3. Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représente les différentes fonctionnalités proposées par le microservice Utilisateur ainsi que les interactions entre les acteurs et le système.

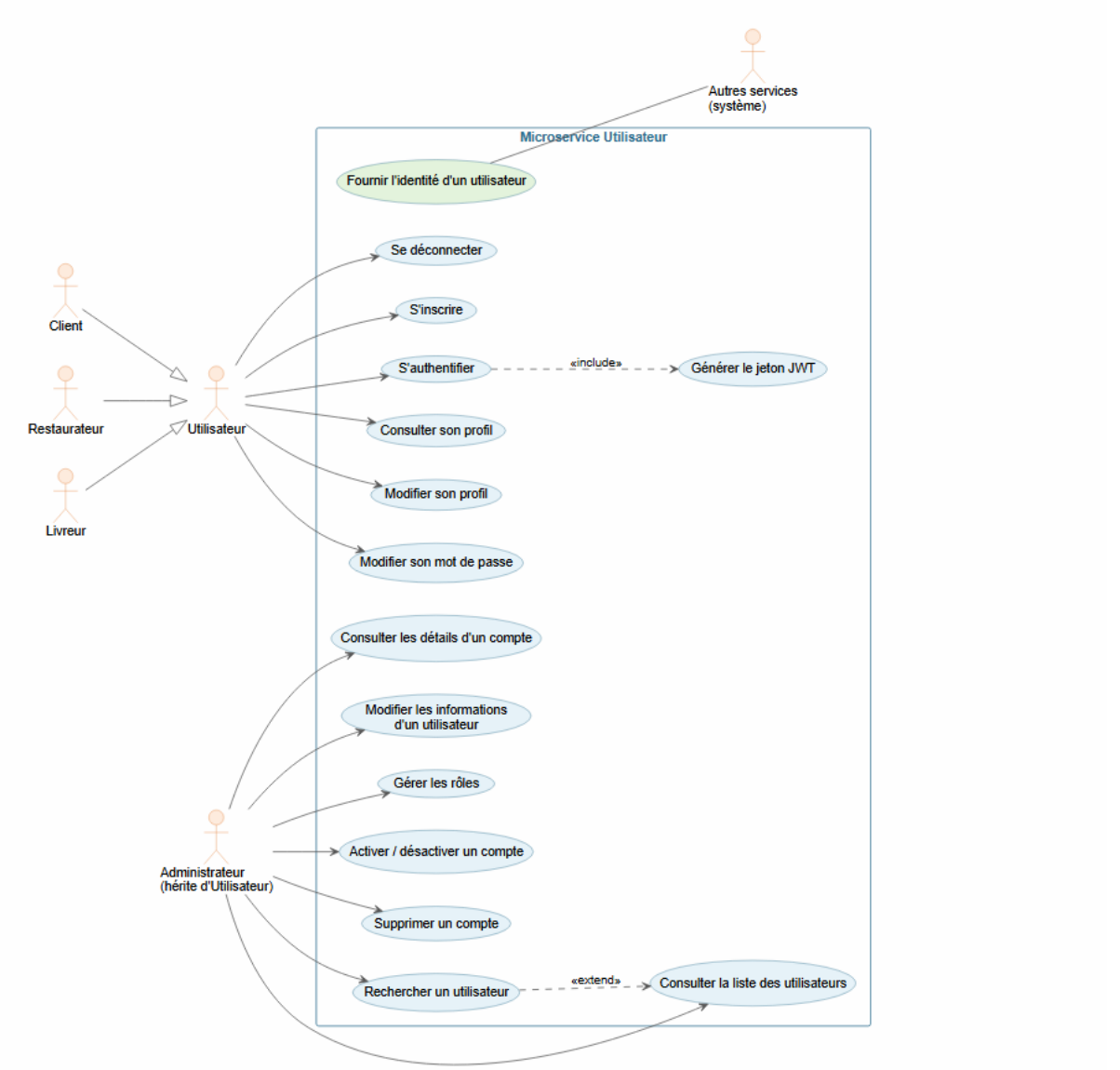


Figure : Diagramme de cas d'utilisation Utilisateur.

3.1 Fonctionnalités du microservice Utilisateur

Le microservice Utilisateur permet de gérer les comptes des différents utilisateurs de la plateforme YallaFood.

Ses principales fonctionnalités sont :

1. Inscription

- Saisir les informations personnelles.

- Saisir une adresse e-mail.
- Définir un mot de passe.
- Vérifier que l'adresse e-mail n'est pas déjà utilisée.
- Enregistrer le nouvel utilisateur dans MySQL.
- Attribuer un rôle à l'utilisateur.

2. Authentification

- Saisir l'adresse e-mail et le mot de passe.
- Vérifier les informations de connexion.
- Vérifier que le compte est actif.
- Générer un token JWT après une authentification réussie.
- Retourner une réponse de confirmation ou un message d'erreur.

3. Consulter le profil

- Récupérer les informations personnelles.
- Afficher le nom, le prénom et l'adresse e-mail.
- Afficher le rôle de l'utilisateur.
- Consulter le statut du compte.

4. Modifier le profil

- Modifier le nom et le prénom.
- Modifier le numéro de téléphone.
- Modifier l'adresse.
- Mettre à jour les informations dans MySQL.

5. Modifier le mot de passe

- Saisir l'ancien mot de passe.
- Vérifier l'ancien mot de passe.
- Saisir un nouveau mot de passe.
- Enregistrer le nouveau mot de passe de manière sécurisée.

6. Gérer les utilisateurs

Cette fonctionnalité est réservée à l'administrateur.

- Afficher la liste des utilisateurs.
- Rechercher un utilisateur par son identifiant ou son adresse e-mail.
- Consulter les détails d'un compte.
- Modifier le rôle d'un utilisateur.
- Activer ou désactiver un compte.
- Supprimer un compte selon les règles définies.

7. Déconnexion

- Terminer la session de l'utilisateur.
- Supprimer le token JWT côté client.

4. Diagramme de classes

Le diagramme de classes présente la structure du microservice Utilisateur ainsi que les relations entre les différentes classes.

Il représente les utilisateurs de la plateforme et leurs rôles.

Les principales classes sont :

- **Utilisateur** : id, nom, prénom, email, motDePasse, telephone, adresse, role, statut, dateCreation, dateMiseAJour.
- **RoleUtilisateur** : CLIENT, RESTAURATEUR, ADMINISTRATEUR, LIVREUR.
- **StatutUtilisateur** : ACTIF, INACTIF, BLOQUE.

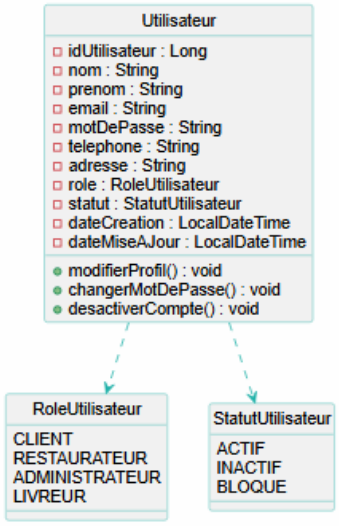


Figure : Diagramme de classes Utilisateur.

5. Diagramme de séquence

Le diagramme de séquence représente le déroulement chronologique des interactions entre les différents acteurs et composants du microservice Utilisateur lors de l'inscription et de l'authentification.

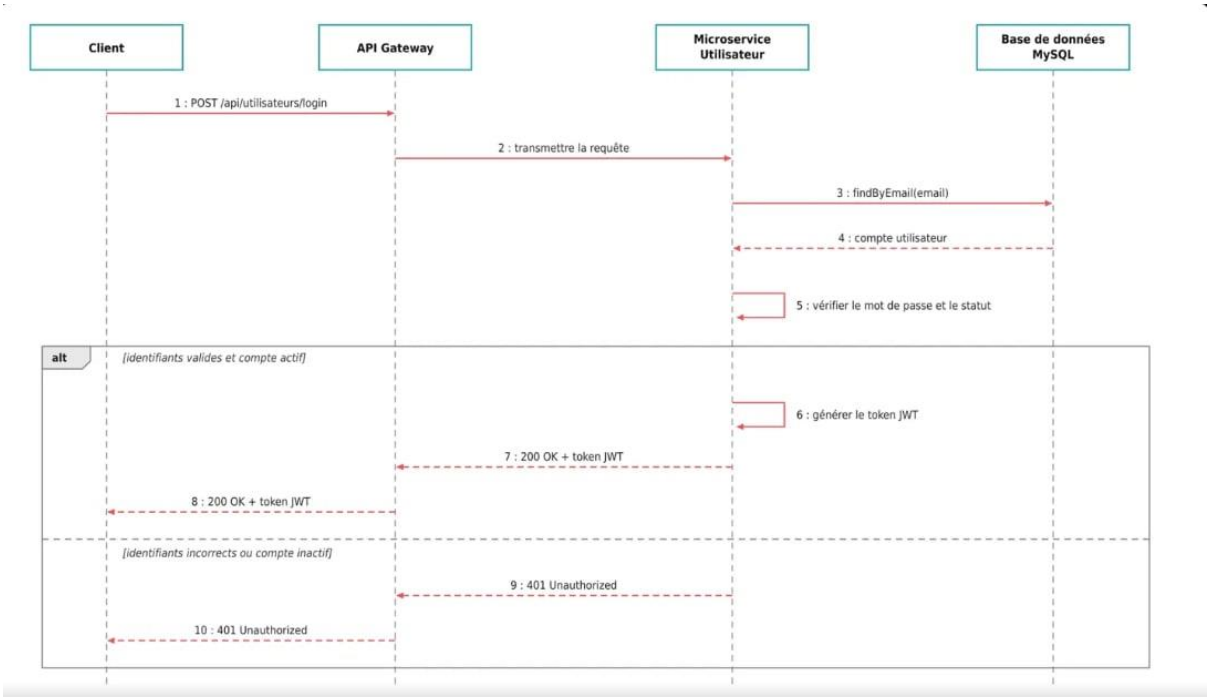


Figure : Diagramme de séquence Utilisateur.

6. Modèle de données et MySQL

Le microservice Utilisateur utilise MySQL comme système de gestion de base de données relationnelle.

Cette base permet de stocker et d'organiser les informations relatives aux comptes des utilisateurs.

Structure de la base de données :

Table	Description
utilisateur	Stocke les informations personnelles, l'adresse e-mail, le mot de passe sécurisé, le rôle et le statut de chaque utilisateur.

Les avantages de MySQL :

- Garantit la cohérence des données.
- Permet de gérer les informations des utilisateurs.
- Facilite les opérations d'ajout, de modification et de suppression.
- Assure la persistance des données.
- Permet de gérer les contraintes d'unicité, notamment sur l'adresse e-mail.

Dans notre architecture, les données des utilisateurs sont gérées indépendamment par le microservice Utilisateur. Les autres microservices utilisent l'identifiant de l'utilisateur comme référence pour identifier le client, le restaurateur, l'administrateur ou le livreur.

7. API REST et endpoints

L'API REST permet aux utilisateurs et aux autres composants de la plateforme de communiquer avec le microservice Utilisateur à travers des requêtes HTTP.

Les échanges de données sont effectués au format JSON.

Méthode HTTP	Endpoint	Fonction

POST	/api/utilisateurs/inscription	Créer un compte
POST	/api/utilisateurs/login	Authentifier un utilisateur
GET	/api/utilisateurs	Afficher la liste des utilisateurs
GET	/api/utilisateurs/{id}	Consulter un utilisateur
GET	/api/utilisateurs/profil	Consulter son profil
PUT	/api/utilisateurs/profil	Modifier son profil
PATCH	/api/utilisateurs/{id}/statut	Activer ou désactiver un compte
PATCH	/api/utilisateurs/{id}/role	Modifier le rôle d'un utilisateur
PATCH	/api/utilisateurs/mot-de-passe	Modifier son mot de passe
DELETE	/api/utilisateurs/{id}	Supprimer un utilisateur

7.1. Les codes de réponse HTTP

Code HTTP	Signification	Description
200	OK	Requête exécutée avec succès
201	Created	Compte créé avec succès
400	Bad Request	Données envoyées invalides
401	Unauthorized	Identifiants incorrects ou token invalide
403	Forbidden	Accès non autorisé
404	Not Found	Utilisateur introuvable
409	Conflict	Adresse e-mail déjà utilisée
500	Internal Server Error	Erreur interne du serveur

7.2. Exemple d'une requête API

Lorsqu'un client souhaite créer un compte, il envoie une requête POST contenant les informations nécessaires.

Requête :

POST /api/utilisateurs/inscription

Content-Type: application/json

```
{
  "nom": "Ben Ali",
  "prenom": "Ahmed",
  "email": "ahmed@gmail.com",
```

```
  "motDePasse": "password123",
  "telephone": "22123456",
  "adresse": "Tunis",
  "role": "CLIENT"
}
```

Réponse :

```
{
  "message": "Compte créé avec succès",
  "idUtilisateur": 25,
  "email": "ahmed@gmail.com",
  "role": "CLIENT",
  "statut": "ACTIF"
}
```

8. Interactions avec les autres microservices

Le microservice Utilisateur ne fonctionne pas seul. Il communique avec les autres microservices pour permettre le fonctionnement global de la plateforme YallaFood.

Interaction avec le microservice Commande :

Le microservice Commande utilise les informations du microservice Utilisateur pour identifier le client qui souhaite passer une commande.

Interaction avec le microservice Restaurant :

Le microservice Restaurant utilise l'identifiant du restaurateur pour associer un restaurant à son propriétaire.

Interaction avec le microservice Livraison :

Le microservice Livraison utilise les informations du livreur pour gérer l'affectation des livraisons.

Interaction avec le microservice Menu :

Le microservice Menu peut utiliser l'identifiant du restaurateur pour identifier le propriétaire du menu.

Microservice	Pourquoi communiquer ?	Données échangées
Commande	Identifier le client	ID utilisateur, identité
Restaurant	Identifier le restaurateur	ID utilisateur
Livraison	Identifier le livreur	ID utilisateur, identité
Menu	Identifier le propriétaire du menu	ID restaurateur

9. Communication synchrone et asynchrone

Dans une architecture microservices, les services peuvent communiquer de deux manières : de façon synchrone ou asynchrone.

A. Communication synchrone

La communication synchrone consiste à envoyer une requête et à attendre directement la réponse du service concerné.

Dans notre projet :

- Le client envoie une requête HTTP pour créer son compte.
- Le client envoie ses identifiants pour se connecter.
- Les autres microservices peuvent demander les informations nécessaires pour identifier un utilisateur.
- L'API Gateway transmet les requêtes au microservice Utilisateur.

Technologie utilisée : HTTP / API REST.

Exemple : récupérer les informations d'un client avant de traiter sa commande.

B. Communication asynchrone

La communication asynchrone permet aux microservices d'échanger des messages sans attendre immédiatement une réponse.

Dans notre projet, RabbitMQ peut être utilisé pour :

- Informer les services concernés lorsqu'un nouvel utilisateur est créé.
- Informer les autres services lorsqu'un compte est désactivé.
- Diffuser les changements importants concernant les utilisateurs.

Technologie utilisée : RabbitMQ.

9.1. Les événements du microservice

Les événements permettent d'informer les autres microservices qu'une action importante s'est produite dans le système.

Les principaux événements liés au microservice Utilisateur sont :

Événement	Description
UtilisateurCreeEvent	Un nouvel utilisateur a été créé
UtilisateurModifieEvent	Les informations d'un utilisateur ont été modifiées
UtilisateurDesactiveEvent	Un compte utilisateur a été désactivé
UtilisateurSupprimeEvent	Un compte utilisateur a été supprimé

Ces événements sont transmis de manière asynchrone à l'aide de RabbitMQ.

10. Conclusion

Le microservice Utilisateur constitue un élément essentiel de l'architecture microservices de YallaFood. Il permet de gérer les comptes, les informations personnelles et l'authentification des différents acteurs de la plateforme.

Grâce à l'utilisation de MySQL, des API REST et de JWT, il garantit une gestion structurée et sécurisée des utilisateurs.

La communication avec les autres microservices et l'utilisation de RabbitMQ facilitent les échanges d'informations et assurent l'indépendance des différents composants de la plateforme.

Cette architecture facilite la maintenance, l'évolution et l'intégration de nouvelles fonctionnalités dans le système.

Présentation du microservice Restaurant

Le microservice Restaurant est un composant indépendant de l'architecture microservices. Il est responsable de la gestion des informations et des disponibilités des restaurants présents sur la plateforme.

Son objectif principal est de permettre aux clients de consulter les restaurants disponibles, de découvrir leurs informations et de faciliter la gestion de leurs établissements par les restaurateurs.

Ses principales responsabilités sont :

- Ajouter de nouveaux restaurants à la plateforme.
- Consulter la liste des restaurants disponibles.
- Gérer les informations des restaurants (nom, adresse, téléphone, description, horaires d'ouverture).
- Modifier les informations d'un restaurant.
- Supprimer ou désactiver un restaurant.
- Gérer les disponibilités des restaurants.
- Fournir les informations nécessaires aux autres microservices.

Technologies utilisées :

- Architecture : Microservices.
- Communication : API REST.
- Base de données : MySQL.
- Format d'échange : JSON.
- Protocole : HTTP.

1. Acteurs et fonctionnalités

Le microservice Restaurant interagit avec différents acteurs afin d'assurer la gestion des établissements présents sur la plateforme.

1. Le Client

Le client est l'acteur qui consulte les restaurants disponibles. Il peut :

- Consulter la liste des restaurants.
- Rechercher un restaurant par nom ou localisation.

- Consulter les informations d'un restaurant.
- Consulter les horaires d'ouverture.
- Consulter les disponibilités des restaurants.

2. Le Restaurateur

Le restaurateur est responsable de la gestion de son établissement. Il peut :

- Ajouter son restaurant à la plateforme.
- Modifier les informations de son restaurant.
- Mettre à jour les horaires d'ouverture.
- Gérer les disponibilités de son restaurant.
- Activer ou désactiver son établissement.

3. Le Microservice Menu

Ce microservice utilise les informations du restaurant pour associer les menus aux établissements.

Ses responsabilités :

- Associer chaque menu à un restaurant.
- Identifier le restaurant auquel appartient un plat.
- Récupérer les informations du restaurant si nécessaire.

5. L'Administrateur

L'administrateur supervise la gestion des restaurants sur la plateforme. Il peut :

- Consulter la liste des restaurants.
- Valider ou refuser l'ajout d'un restaurant.
- Modifier ou supprimer un restaurant.
- Désactiver un établissement en cas de problème.
- Superviser les informations des restaurants.

2.Interaction du microservice Restaurant avec les autres microservices :

Le microservice Restaurant interagit avec plusieurs microservices pour assurer le bon fonctionnement de la plateforme. Il communique avec :

- le microservice Menu afin d'associer les menus aux restaurants et de faciliter leur consultation.
- le microservice Commande et Livraison pour fournir les informations nécessaires au traitement des commandes.
- le microservice Utilisateur pour vérifier l'identité et les droits des restaurateurs qui souhaitent gérer leurs établissements.

Diagramme de cas d'utilisation :

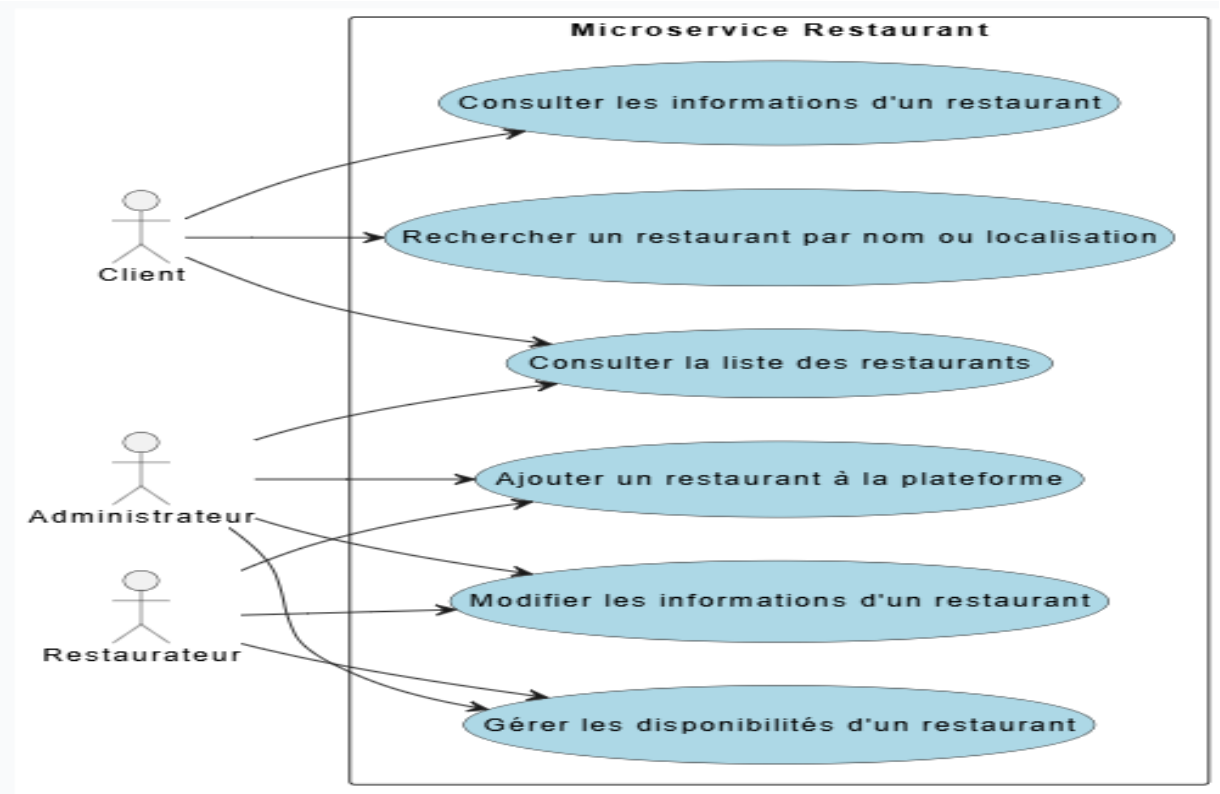


Diagramme de classe :

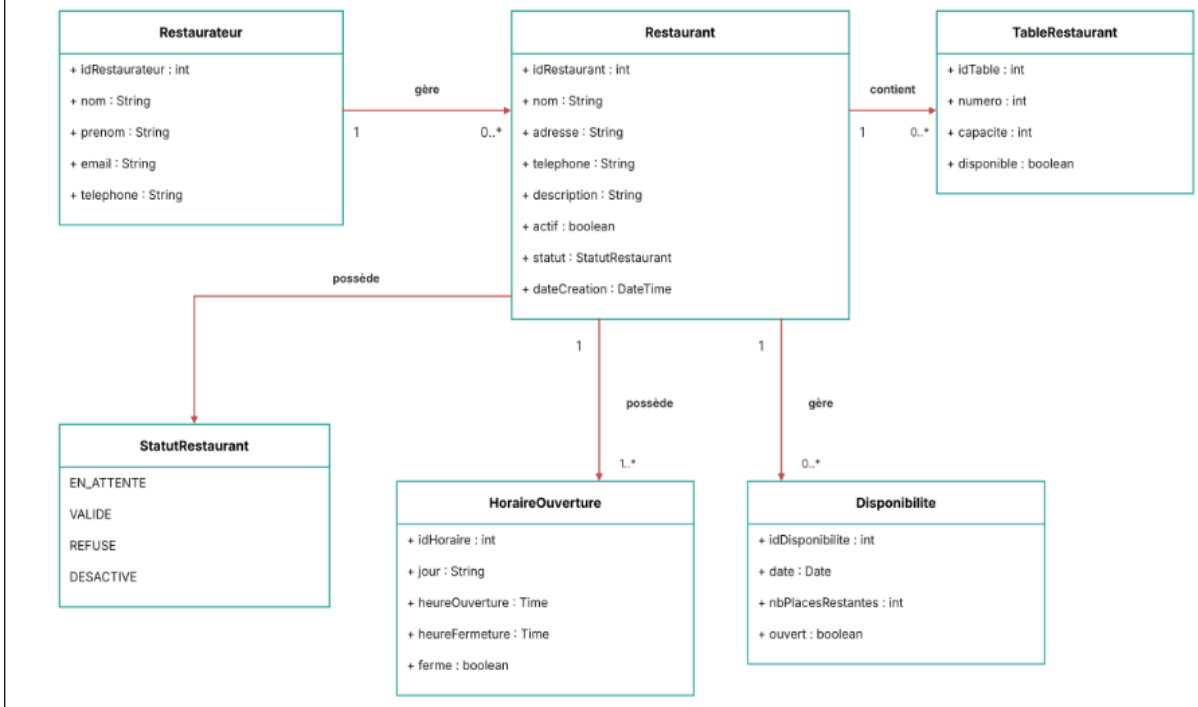
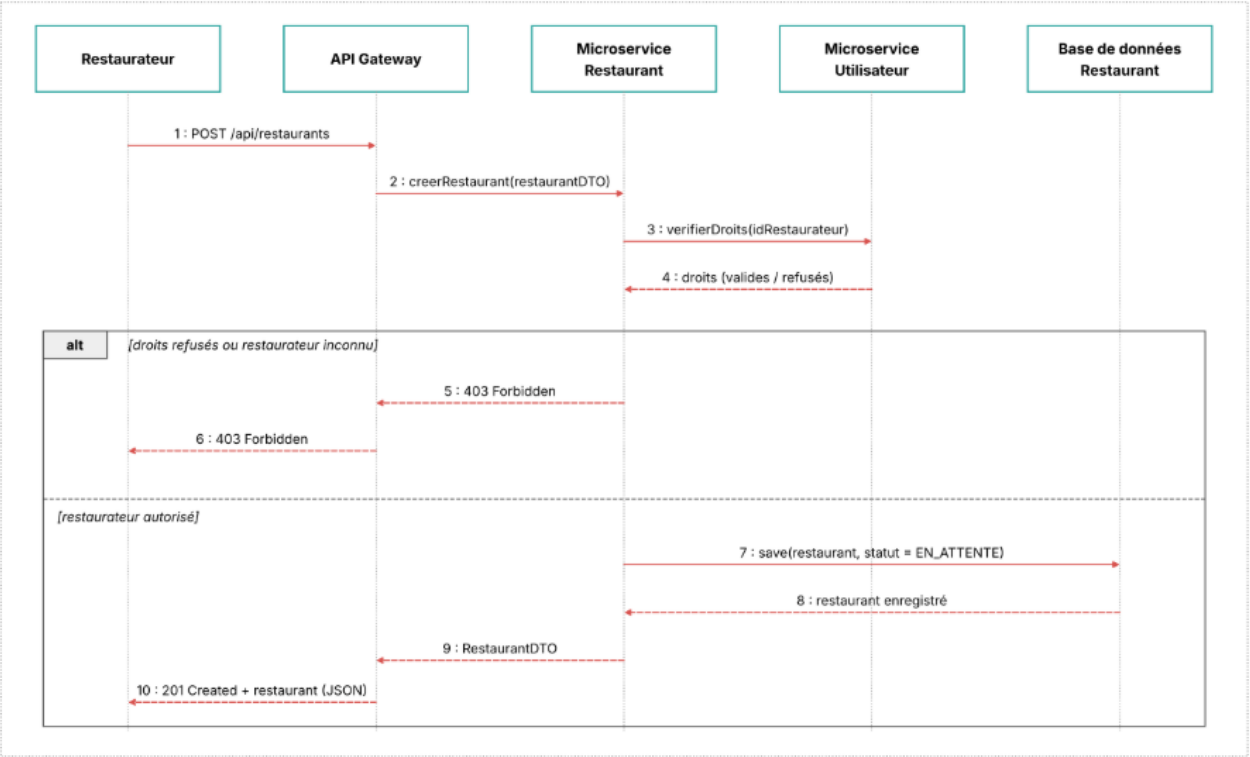


Diagramme de séquence :



3.Les fonctionnalités du microservice Restaurant

Le microservice Restaurant offre plusieurs fonctionnalités permettant d'assurer la gestion des restaurants présents sur la plateforme. Ces fonctionnalités facilitent la gestion des établissements par les restaurateurs et permettent aux clients de consulter les restaurants avant de réserver ou de commander.

Les principales fonctionnalités sont :

1. Gestion des restaurants

- Ajouter un nouveau restaurant à la plateforme.
- Consulter la liste des restaurants disponibles.
- Afficher les détails d'un restaurant (nom, adresse, téléphone, description).
- Modifier les informations d'un restaurant.
- Supprimer ou désactiver un restaurant.

2. Gestion des horaires d'ouverture

- Définir les horaires d'ouverture de chaque restaurant.
- Mettre à jour les horaires d'ouverture selon les besoins du restaurateur.
- Afficher les horaires d'ouverture aux clients lors de la consultation.

3. Gestion de la disponibilité

- Indiquer si un restaurant est disponible ou indisponible.
- Gérer la disponibilité des tables d'un restaurant.
- Modifier l'état de disponibilité d'un restaurant.
- Permettre aux clients de consulter uniquement les restaurants disponibles.

4. Gestion de l'état d'un restaurant

- Activer ou désactiver un établissement.
- Permettre à l'administrateur de valider ou de refuser l'ajout d'un restaurant.
- Permettre à l'administrateur de désactiver un établissement en cas de problème.

5. Consultation et recherche

- Consulter la liste des restaurants.
- Rechercher un restaurant par nom ou par localisation.
- Consulter les informations détaillées d'un restaurant.
- Consulter les horaires d'ouverture et les disponibilités.

6. Supervision par l'administrateur

- Consulter la liste de tous les restaurants.
- Valider ou refuser l'ajout d'un restaurant.
- Modifier ou supprimer un restaurant.
- Superviser les informations des restaurants.

7. Association avec les autres microservices

- **Menu** : permettre d'associer chaque menu à un restaurant et d'identifier le restaurant auquel appartient un plat.
- **Commande et Livraison** : fournir les informations nécessaires au traitement des commandes.
- **Utilisateur** : vérifier l'identité et les droits des restaurateurs qui souhaitent gérer leurs établissements.

4.API REST et endpoints :

Methode Http	Endpoint	Fonction
POST	`/api/restaurants`	Ajouter un restaurant
GET	`/api/restaurants`	Afficher les restaurants
GET	`/api/restaurants/{id}`	Consulter un restaurant
PUT	`/api/restaurants/{id}`	Modifier un restaurant
DELETE	`/api/restaurants/{id}`	Supprimer un restaurant
GET	`/api/restaurants/{id}/disponibilites`	Vérifier les disponibilités

Le tableau ci-dessus présente les principaux endpoints de l'API REST du microservice Restaurant. Il indique pour chaque opération la méthode HTTP utilisée, l'URL d'accès et la fonction réalisée.

Ces endpoints permettent de gérer le cycle de vie d'un restaurant : l'ajouter (POST), afficher la liste (GET), consulter un restaurant précis (GET avec `{id}`), modifier ses informations (PUT) et le supprimer (DELETE). Le dernier endpoint, `/api/restaurants/{id}/disponibilites`, permet de vérifier la disponibilité d'un restaurant. Il est principalement utilisé par le microservice Réservation.

5.Modèle de données et MySQL

Le microservice Restaurant utilise MySQL comme système de gestion de base de données relationnelle.

Cette base permet de stocker et d'organiser les informations relatives aux restaurants, à leurs horaires d'ouverture, à leurs tables et à leurs disponibilités.

Tableau – Structure de la base de données (microservice Restaurant)

Table	Description
restaurant	Stocke les informations des restaurants : nom, adresse, téléphone, description, statut (EN_ATTENTE, VALIDE, REFUSE, DESACTIVE), état actif, identifiant du restaurateur et date de création
horaire_ouverture	Contient les horaires d'ouverture de chaque restaurant : jour, heure d'ouverture, heure de fermeture et jour de fermeture
disponibilite	Contient les disponibilités d'un restaurant par date : nombre de places restantes et état ouvert ou fermé
table_restaurant	Contient les tables de chaque restaurant : numéro, capacité et état de disponibilité

Chaque restaurant possède plusieurs horaires, plusieurs disponibilités et plusieurs tables. Les clés étrangères `idRestaurant` relient ces trois tables à la table `restaurant`.

Les avantages de MySQL :

- Garantit la cohérence des données.
- Assure la sécurité des transactions.
- Permet de gérer les relations entre les données (Restaurant – Horaires – Disponibilités – Tables).
- Facilite les opérations d'ajout, de modification et de suppression.

Dans notre architecture, les données des restaurants sont gérées indépendamment par le microservice Restaurant. L'identifiant du restaurateur (`idRestaurateur`) est une simple référence vers le microservice Utilisateur : le microservice Restaurant ne stocke aucune autre information sur l'utilisateur.

6.Communication synchrone et asynchrone

Dans une architecture microservices, les services peuvent communiquer de deux manières : de façon synchrone ou asynchrone.

A. Communication synchrone

La communication synchrone consiste à envoyer une requête et à attendre directement la réponse du service concerné.

Dans notre projet :

- Le microservice Menu interroge le microservice Restaurant pour vérifier qu'un restaurant existe et est actif avant d'y associer un menu.
- Le microservice Livraison interroge le microservice Restaurant pour obtenir l'adresse de retrait du repas.
- Le microservice Restaurant interroge le microservice Utilisateur pour vérifier l'identité et les droits d'un restaurateur avant d'ajouter ou de modifier un établissement.

Tableau – Communication synchrone (REST)

Type	Interaction	Objectif
Synchrone (REST)	Réservation → Restaurant : GET /api/restaurants/{id}/disponibilites	Vérifier les horaires et la disponibilité des tables
Synchrone (REST)	Menu → Restaurant : GET /api/restaurants/{id}	Vérifier l'existence et l'état du restaurant auquel un menu est associé
Synchrone (REST)	Livraison → Restaurant : GET /api/restaurants/{id}	Récupérer l'adresse de retrait du restaurant
Synchrone (REST)	Restaurant → Utilisateur : vérification des droits	Contrôler l'identité et les droits du restaurateur

Exemple : vérifier immédiatement si un restaurant est ouvert et dispose d'une table libre avant de confirmer une réservation.

B. Communication asynchrone

La communication asynchrone permet aux microservices d'échanger des messages sans attendre immédiatement une réponse.

Dans notre projet, elle est utilisée pour :

- Informer les autres services qu'un restaurant a été désactivé ou que ses informations ont changé.
- Informer le restaurateur lorsque l'administrateur valide ou refuse son restaurant.
- Transmettre les changements de disponibilité d'un restaurant.

Technologie utilisée : RabbitMQ.

Tableau – Communication asynchrone (événements)

Type	Interaction	Objectif
Asynchrone (événement)	Restaurant → Menu : RestaurantDeactivated	Masquer les menus d'un restaurant désactivé
Asynchrone (événement)	Restaurant → Réservation : RestaurantDeactivated, DisponibiliteUpdated	Bloquer les nouvelles réservations d'un restaurant désactivé et tenir compte des disponibilités
Asynchrone (événement)	Restaurant → Notification : RestaurantValidated, RestaurantRejected	Prévenir le restaurateur de la décision de l'administrateur

. Les événements du microservice

Les événements permettent d'informer les autres microservices qu'une action importante s'est produite dans le système.

Tableau – Les événements du microservice Restaurant

Événement	Description	Consommateurs
RestaurantCreated	Un nouveau restaurant a été ajouté et attend la validation de l'administrateur	Notification
RestaurantValidated	L'administrateur a validé le restaurant	Notification, Menu
RestaurantRejected	L'administrateur a refusé le restaurant	Notification
RestaurantUpdated	Les informations ou les horaires d'un restaurant ont été modifiés	Menu, Réservation
RestaurantDeactivated	Un restaurant a été désactivé par le restaurateur ou l'administrateur	Menu, Réservation, Commande
DisponibiliteUpdated	La disponibilité d'un restaurant a été modifiée	Réservation

Conclusion

Le microservice Restaurant est un élément essentiel de notre architecture microservices. Il permet de gérer efficacement les restaurants présents sur la plateforme : ajout, modification, horaires d'ouverture, disponibilités, activation et

désactivation. Il fournit aussi aux autres microservices les informations dont ils ont besoin.

Grâce à l'utilisation de MySQL, des API REST et de la communication entre microservices, il garantit une gestion structurée et indépendante des données des restaurants. La communication synchrone permet de vérifier immédiatement les informations d'un restaurant, par exemple ses horaires ou sa disponibilité pour une réservation. La communication asynchrone, par événements, permet d'informer les autres services d'un changement important, comme la validation ou la désactivation d'un restaurant, sans les bloquer.

Cette architecture facilite la maintenance, l'évolution et l'intégration de nouvelles fonctionnalités dans le système.

2.Présentation du microservice Menu

Le microservice Menu est un composant indépendant de l'architecture microservices. Il est responsable de la gestion des menus et des plats proposés par les différents restaurants présents sur la plateforme.

Son objectif principal est de permettre aux clients de consulter les menus des restaurants, de découvrir les plats disponibles ainsi que leurs prix et leurs descriptions, tout en facilitant la gestion des menus par les restaurateurs.

Ses principales responsabilités sont :

- Ajouter de nouveaux menus pour les restaurants.
- Consulter la liste des menus disponibles.
- Gérer les informations des menus (nom, description, restaurant associé).
- Ajouter des plats aux menus.
- Modifier les informations des plats (nom, description, prix, image).
- Supprimer ou désactiver un plat ou un menu.
- Gérer la disponibilité des plats.
- Fournir les informations nécessaires aux autres microservices, notamment Restaurant et Commande.

Technologies utilisées :

- Architecture : Microservices.
- Communication : API REST.
- Base de données : MySQL.
- Format d'échange : JSON.
- Protocole : HTTP.

2.1.Acteurs et fonctionnalités du microservice Menu

Le microservice Menu intervient dans la gestion des menus et des plats proposés par les différents restaurants de la plateforme. Il permet aux restaurateurs de gérer leurs offres et aux clients de consulter les plats disponibles afin de choisir les repas qui répondent à leurs préférences.

Ce microservice met à disposition plusieurs fonctionnalités accessibles selon le rôle de chaque acteur.

- **Les acteurs**

1. Administrateur

L'administrateur est responsable de la supervision et du contrôle des menus présents sur la plateforme. Il veille au bon fonctionnement du service et à la cohérence des informations enregistrées.

Ses principales responsabilités sont :

- Consulter l'ensemble des menus et des plats enregistrés.

- Ajouter, modifier ou supprimer des menus si nécessaire.
- Contrôler les informations des menus et des plats.
- Activer ou désactiver un menu ou un plat.
- Assurer la gestion globale des données liées aux menus.

2. Restaurateur (Responsable du restaurant)

Le restaurateur est l'acteur principal dans la gestion des menus de son établissement. Il peut créer et organiser les menus, ajouter de nouveaux plats et mettre à jour leurs informations selon les offres proposées par son restaurant.

Ses principales responsabilités sont :

- Créer un nouveau menu pour son restaurant.
- Consulter les menus de son établissement.
- Modifier les informations d'un menu (nom, description, etc.).
- Ajouter des plats à un menu.
- Modifier les informations des plats (nom, description, prix et image).
- Supprimer un plat ou un menu.
- Gérer la disponibilité des plats selon les stocks ou les offres du restaurant.

3. Client

Le client est l'utilisateur qui consulte les menus des différents restaurants afin de découvrir les repas proposés et de choisir les plats qu'il souhaite commander.

Ses principales responsabilités sont :

- Consulter les menus des restaurants disponibles.
- Afficher les détails des différents plats.
- Consulter les descriptions, les prix et les images des plats.
- Vérifier la disponibilité des plats.
- Choisir les plats qu'il souhaite commander.
- Accéder aux informations nécessaires pour effectuer une commande.

Les fonctionnalités du microservice Menu

Le microservice Menu offre plusieurs fonctionnalités permettant d'assurer la gestion des menus et des plats proposés par les différents restaurants de la plateforme. Ces

fonctionnalités facilitent la gestion des offres par les restaurateurs et permettent aux clients de consulter les repas disponibles avant de passer une commande.

Les principales fonctionnalités sont :

1. Gestion des menus

- Ajouter un nouveau menu associé à un restaurant.
- Consulter la liste des menus disponibles.
- Afficher les détails d'un menu (nom, description et restaurant associé).
- Modifier les informations d'un menu.
- Supprimer ou désactiver un menu.

2. Gestion des plats

- Ajouter de nouveaux plats à un menu.
- Consulter la liste des plats proposés.
- Afficher les détails de chaque plat (nom, description, prix et image).
- Modifier les informations d'un plat.
- Supprimer un plat d'un menu.

3. Gestion des prix

- Définir le prix de chaque plat.
- Modifier les prix selon les besoins du restaurant.
- Afficher les prix des plats aux clients lors de la consultation du menu.

4. Gestion de la disponibilité

- Indiquer si un plat est disponible ou indisponible.
- Modifier l'état de disponibilité d'un plat.
- Permettre aux clients de consulter uniquement les plats pouvant être commandés.

5. Consultation et recherche

- Consulter les menus des différents restaurants.
- Rechercher des plats selon leur nom.
- Filtrer les plats selon certains critères.
- Consulter les informations détaillées des menus et des plats.

6. Association avec les autres microservices

- Associer chaque menu au restaurant correspondant.
- Fournir les informations des plats au microservice Commande.
- Transmettre les informations nécessaires concernant les menus et leur disponibilité aux autres microservices concernés.
-

2.2.Interaction avec les autres microservices

Le microservice Menu communique avec les autres composants de la plateforme afin d'assurer la cohérence des informations :

- Microservice Restaurant : récupère les informations des restaurants pour associer chaque menu à son établissement.

- Microservice Commande : fournit les informations des plats, notamment leurs noms, leurs prix et leur disponibilité, afin de permettre aux clients de passer des commandes.

- Microservice Réservation : peut fournir les informations relatives aux menus proposés par les restaurants lors de la consultation ou de la réservation.

Diagramme de cas d'utilisation

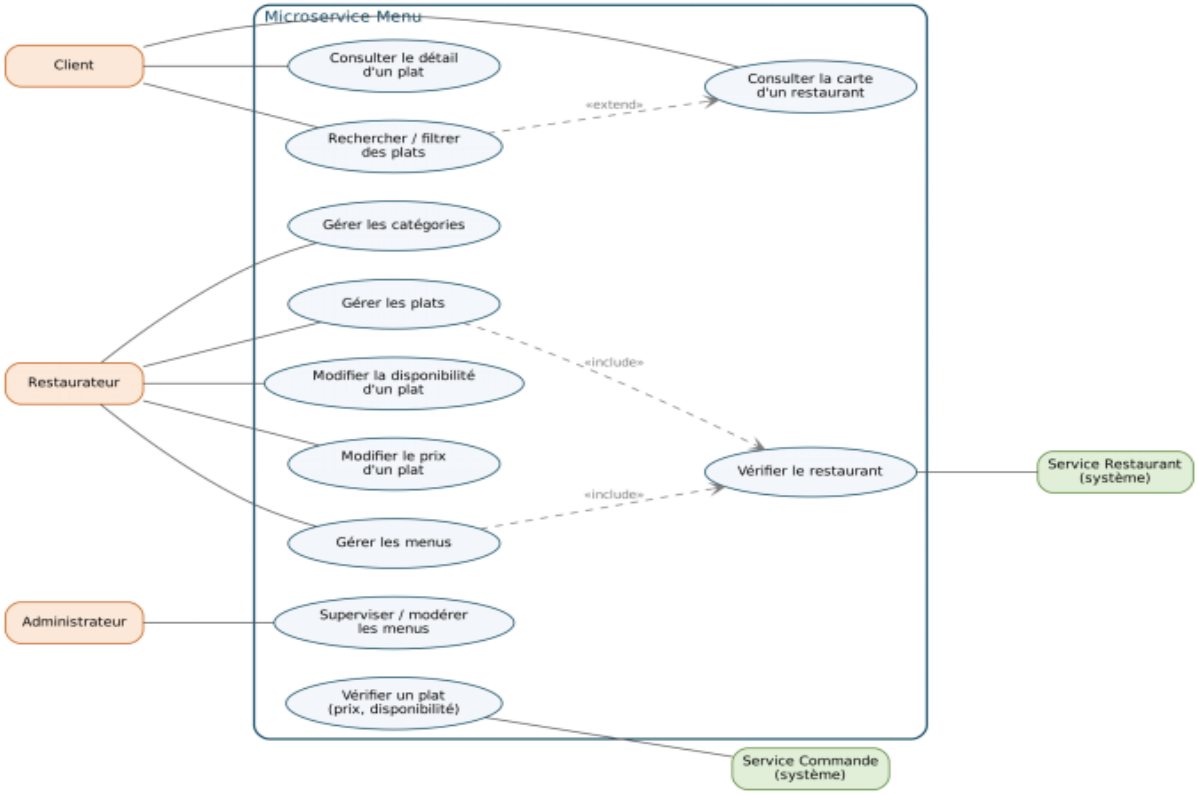


Figure : Digramme de cas d'utilisation de menu

Diagramme de classe :

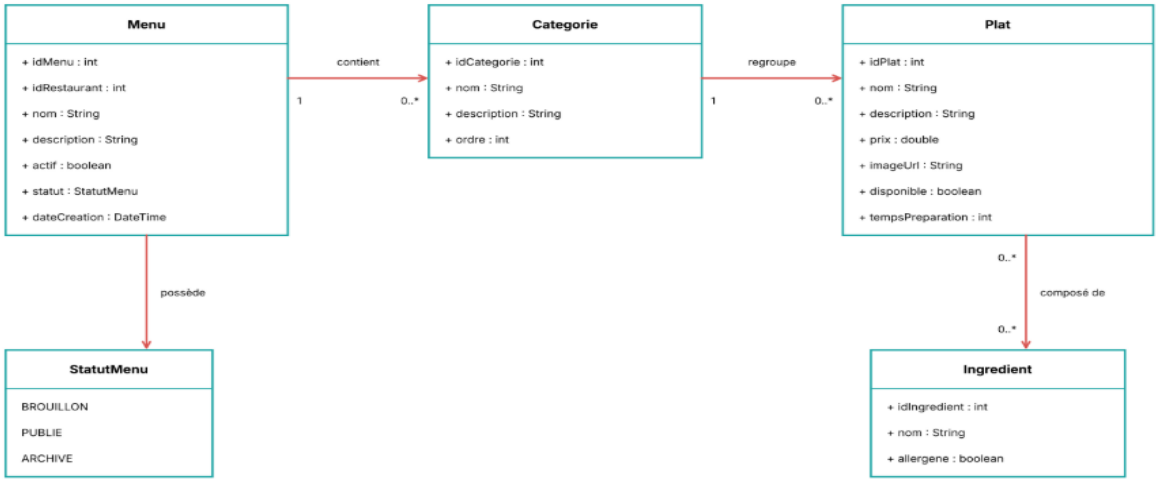


Figure : Digramme de classe menu

Diagramme de séquence

Le diagramme de séquence représente le déroulement chronologique des interactions entre les différents acteurs et composants du menu lors de la création d'un menu

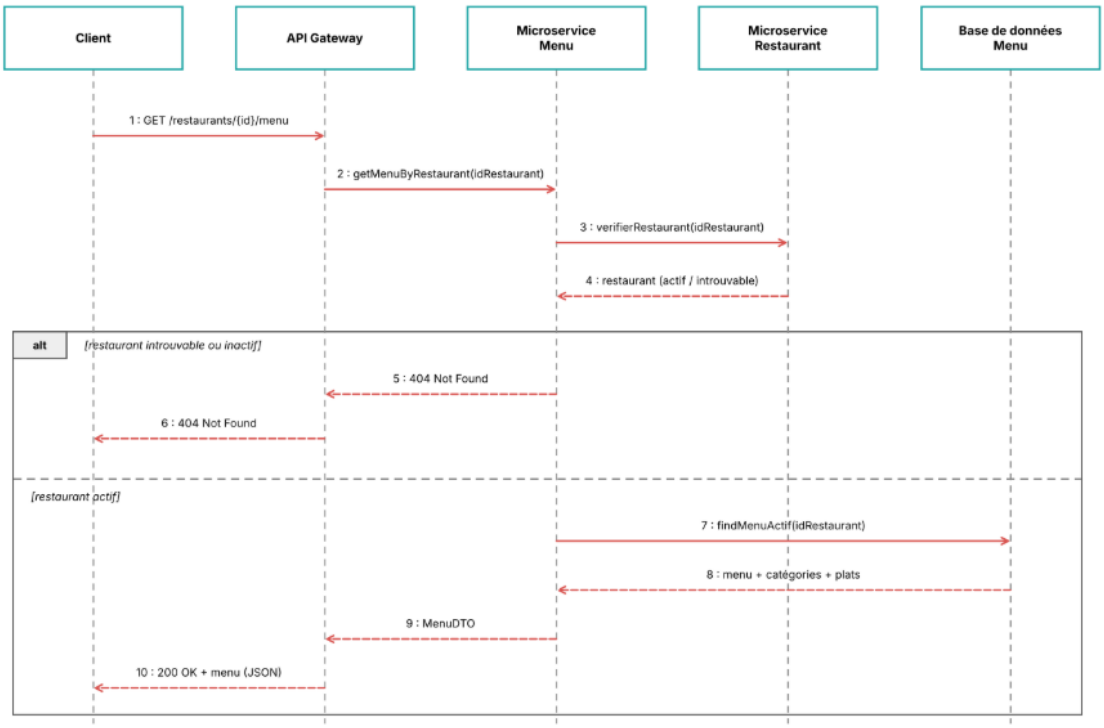


Figure : Digramme de séquence de menu

API REST et endpoints

8.1 Menus

Méthode	Endpoint	Description	Codes de réponse
POST	/api/menus	Créer un menu	201 Created, 400, 404 Restaurant introuvable, 409 Nom déjà utilisé
GET	/api/menus?restaurantId=&statut=	Lister les menus (filtres)	200 OK
GET	/api/menus/{id}	Détails d'un menu	200 OK, 404 Not Found
GET	/api/menus/{id}/carte	Carte complète (catégories et plats)	200 OK, 404
PUT	/api/menus/{id}	Modifier un menu	200 OK, 400, 404
PATCH	/api/menus/{id}/statut	Activer / désactiver un menu	200 OK, 404
DELETE	/api/menus/{id}	Désactiver un menu (suppression logique)	204 No Content, 404

8.2 Catégories

Méthode	Endpoint	Description	Codes de réponse
POST	/api/menus/{id}/categories	Ajouter une catégorie à un menu	201 Created, 400, 404 Menu introuvable
GET	/api/menus/{id}/categories	Lister les catégories d'un menu	200 OK, 404
GET	/api/categories/{id}	Détails d'une catégorie	200 OK, 404 Not Found
PUT	/api/categories/{id}	Modifier une catégorie	200 OK, 400, 404
DELETE	/api/categories/{id}	Supprimer une catégorie	204 No Content, 404

8.3 Plats

Méthode	Endpoint	Description	Codes de réponse
POST	/api/categories/{id}/plats	Ajouter un plat à une catégorie	201 Created, 400, 404 Catégorie introuvable
GET	/api/plats?categorieId=&disponible=	Lister les plats (filtres)	200 OK
GET	/api/plats/{id}	Détails d'un plat	200 OK, 404 Not Found
PUT	/api/plats/{id}	Modifier un plat	200 OK, 400, 404
PATCH	/api/plats/{id}/disponibilite	Activer / désactiver la disponibilité d'un plat	200 OK, 404
POST	/api/plats/{id}/ingredients/{idIngredient}	Associer un ingrédient à un plat	200 OK, 404, 409 Déjà associé
DELETE	/api/plats/{id}/ingredients/{idIngredient}	Retirer un ingrédient d'un plat	204 No Content, 404
DELETE	/api/plats/{id}	Supprimer un plat	204 No Content, 404

8.4 Ingrédients

Méthode	Endpoint	Description	Codes de réponse
POST	/api/ingredients	Créer un ingrédient	201 Created, 400, 409 Nom déjà utilisé
GET	/api/ingredients	Lister les ingrédients	200 OK
GET	/api/ingredients/{id}	Détails d'un ingrédient	200 OK, 404 Not Found
PUT	/api/ingredients/{id}	Modifier un ingrédient	200 OK, 400, 404
DELETE	/api/ingredients/{id}	Supprimer un ingrédient	204 No Content, 404

Le microservice Menu expose une API REST qui permet aux autres services et aux clients, via l'API Gateway, de gérer et de consulter les menus. Elle repose sur le protocole HTTP et échange les données au format JSON. Chaque ressource (menu,

catégorie, plat, ingrédient) est accessible par un endpoint, c'est-à-dire une URL associée à une méthode HTTP : GET pour consulter, POST pour créer, PUT pour modifier, PATCH pour changer un état (statut d'un menu, disponibilité d'un plat) et DELETE pour supprimer. Chaque réponse renvoie un code HTTP standard (200 OK, 201 Created, 204 No Content, 400, 404 Not Found, 409) qui indique le résultat de l'opération. Les endpoints sont regroupés en quatre parties : les menus, les catégories, les plats et les ingrédients.

5. Modèle de données et MySQL

Le microservice Menu utilise MySQL comme système de gestion de base de données relationnelle.

Cette base permet de stocker et d'organiser les informations relatives aux menus des restaurants, à leurs catégories, à leurs plats et aux ingrédients qui composent ces plats. Chaque menu possède plusieurs catégories, et chaque catégorie regroupe plusieurs plats. La clé étrangère idMenu relie la table categorie à la table menu, et la clé étrangère idCategorie relie la table plat à la table categorie. Un plat peut contenir plusieurs ingrédients, et un ingrédient peut être utilisé dans plusieurs plats : cette relation est gérée par la table d'association plat_ingredient.

Tableau – Structure de la base de données (microservice Menu)

Table	Description
menu	Stocke les menus des restaurants : nom, description, statut (BROUILLON, PUBLIE, ARCHIVE), état actif, identifiant du restaurant (idRestaurant) et date de création
categorie	Contient les catégories de chaque menu (entrées, plats principaux, desserts...) : nom, description et ordre d'affichage
plat	Contient les plats de chaque catégorie : nom, description, prix, image, état de disponibilité et temps de préparation
ingredient	Contient la liste des ingrédients : nom et indicateur d'allergène
plat_ingredient	Table d'association entre plat et ingredient : indique les ingrédients qui composent chaque plat

Les avantages de MySQL :

- Garantit la cohérence des données.
- Assure la sécurité des transactions.
- Permet de gérer les relations entre les données (Menu – Catégories – Plats – Ingrédients).
- Facilite les opérations d'ajout, de modification et de suppression.

Dans notre architecture, les données des menus sont gérées indépendamment par le microservice Menu. L'identifiant du restaurant (idRestaurant) est une simple référence vers le microservice Restaurant : le microservice Menu ne stocke aucune autre information sur le restaurant.

6. Communication synchrone et asynchrone

Dans une architecture microservices, les services peuvent communiquer de deux manières : de façon synchrone ou asynchrone.

A. Communication synchrone

La communication synchrone consiste à envoyer une requête et à attendre directement la réponse du service concerné.

Dans notre projet :

- Le microservice Menu interroge le microservice Restaurant pour vérifier qu'un restaurant existe et est actif avant de créer ou de publier un menu qui lui est associé.

- Le microservice Menu est interrogé par le client, via l'API Gateway, pour consulter le menu actif d'un restaurant (catégories et plats).

- Le microservice Livraison interroge le microservice Menu pour obtenir le nom, le prix et la disponibilité d'un plat.

Tableau – Communication synchrone (REST)

Type	Interaction	Objectif
Synchrone (REST)	Menu → Restaurant : GET /api/restaurants/{id}	Vérifier l'existence et l'état du restaurant avant de créer ou de publier un menu
Synchrone (REST)	Client (via API Gateway) → Menu : GET /api/restaurants/{id}/menu	Consulter le menu actif d'un restaurant (catégories et plats)

B. Communication asynchrone

La communication asynchrone permet aux microservices d'échanger des messages sans attendre immédiatement une réponse.

Dans notre projet, elle est utilisée pour :

- Informer le microservice Menu lorsqu'un restaurant a été désactivé, afin de masquer les menus de ce restaurant.
- Informer les autres services lorsque la disponibilité d'un plat change (par exemple un plat devenu indisponible).
- Informer le restaurateur lorsque son menu est publié ou archivé.

Technologie utilisée : RabbitMQ

Tableau – Communication asynchrone (événements)

Type	Interaction	Objectif
Asynchrone (événement)	Restaurant → Menu : RestaurantDeactivated	Masquer les menus d'un restaurant désactivé
Asynchrone (événement)	Menu → Livraison : PlatAvailabilityUpdated	Tenir compte des plats devenus indisponibles
Asynchrone (événement)	Menu → Notification : MenuPublished, MenuArchived	Prévenir le restaurateur de la publication ou de l'archivage de son menu

.

Les événements du microservice Menu

Les événements permettent d'informer les autres microservices qu'une action importante s'est produite dans le système. Le microservice Menu publie ses événements via RabbitMQ lorsqu'un menu change d'état ou que les informations d'un plat sont modifiées.

Tableau – Les événements du microservice Menu

Événement	Description	Consommateurs
MenuPublished	Un menu a été publié et est désormais visible par les clients	Notification, Commande
MenuArchived	Un menu a été archivé et n'est plus proposé aux clients	Notification, Commande
PlatAvailabilityUpdated	La disponibilité d'un plat a été modifiée (disponible ou indisponible)	Commande
PlatPriceUpdated	Le prix d'un plat a été modifié	Commande
PlatDeleted	Un plat a été supprimé d'une catégorie du menu	Commande

Conclusion

Le microservice Menu joue un rôle central dans notre plateforme de réservation et de commande de repas : il permet aux restaurants de présenter leur offre (menus, catégories et plats) et aux clients de la consulter facilement avant de réserver ou de commander.

Grâce à son architecture indépendante, avec sa propre base de données et sa propre API, il peut évoluer sans perturber les autres services. Sa communication avec les microservices Restaurant et Commande, par appels REST et par événements, garantit la cohérence des informations dans toute la plateforme.

3. Présentation du microservice Commande

Le microservice Commande est un composant indépendant de l'architecture microservices. Il est responsable de la gestion des commandes de repas des clients auprès des restaurants.

Son objectif principal est de permettre aux clients de commander des plats, à emporter ou en livraison, et de suivre leur commande jusqu'à la remise du repas.

Ses principales responsabilités sont :

- Recevoir les commandes des clients.
- Vérifier l'existence, le prix et la disponibilité des plats auprès du microservice Menu.
- Vérifier que le restaurant existe et est actif auprès du microservice Restaurant.
- Enregistrer les commandes dans une base de données MySQL.
- Gérer le statut des commandes.
- Permettre la consultation et l'annulation des commandes.
- Retourner une confirmation ou un message de refus au client.
- Informer les services concernés (Livraison, Notification) des changements de statut.

Technologies utilisées :

- Architecture : Microservices.
- Communication : API REST.
- Base de données : MySQL.
- Format d'échange : JSON.
- Protocole : HTTP.

4. Acteurs et fonctionnalités

Le microservice Commande interagit avec différents acteurs afin d'assurer le bon fonctionnement du processus de commande.

1. Le Client

Le client est l'acteur principal du système. Il peut :

- Passer une nouvelle commande.
- Consulter les détails de sa commande.
- Suivre le statut de sa commande.
- Annuler une commande.
- Recevoir une confirmation ou un message de refus.

2. Le Restaurateur

Le restaurateur traite les commandes reçues par son restaurant. Il peut :

- Consulter les commandes de son restaurant.
- Confirmer ou refuser une commande.
- Indiquer qu'une commande est en préparation, puis prête.
- Marquer une commande à emporter comme remise au client.

3. Le Microservice Menu

Ce microservice fournit les informations nécessaires sur les plats.

Ses responsabilités :

- Fournir le nom et le prix des plats.
- Vérifier la disponibilité des plats.
- Retourner le résultat de la vérification au microservice Commande.

4. Le Microservice Restaurant

Ce microservice permet de vérifier que le restaurant existe et est actif.

Ses responsabilités :

- Fournir les informations du restaurant.
- Retourner le résultat de la vérification au microservice Commande.

5. Le Microservice Livraison

Ce microservice prend en charge la commande confirmée en mode livraison.

Ses responsabilités :

- Créer la livraison et assigner un livreur.
- Informer le microservice Commande de l'avancement de la livraison.

6. Le Service Notification

Ce service prévient le client ou le restaurateur à chaque changement de statut d'une commande.

7. L'Administrateur

L'administrateur peut consulter et superviser les commandes, selon les droits définis dans le système.

.

5. Diagramme de cas d'utilisation

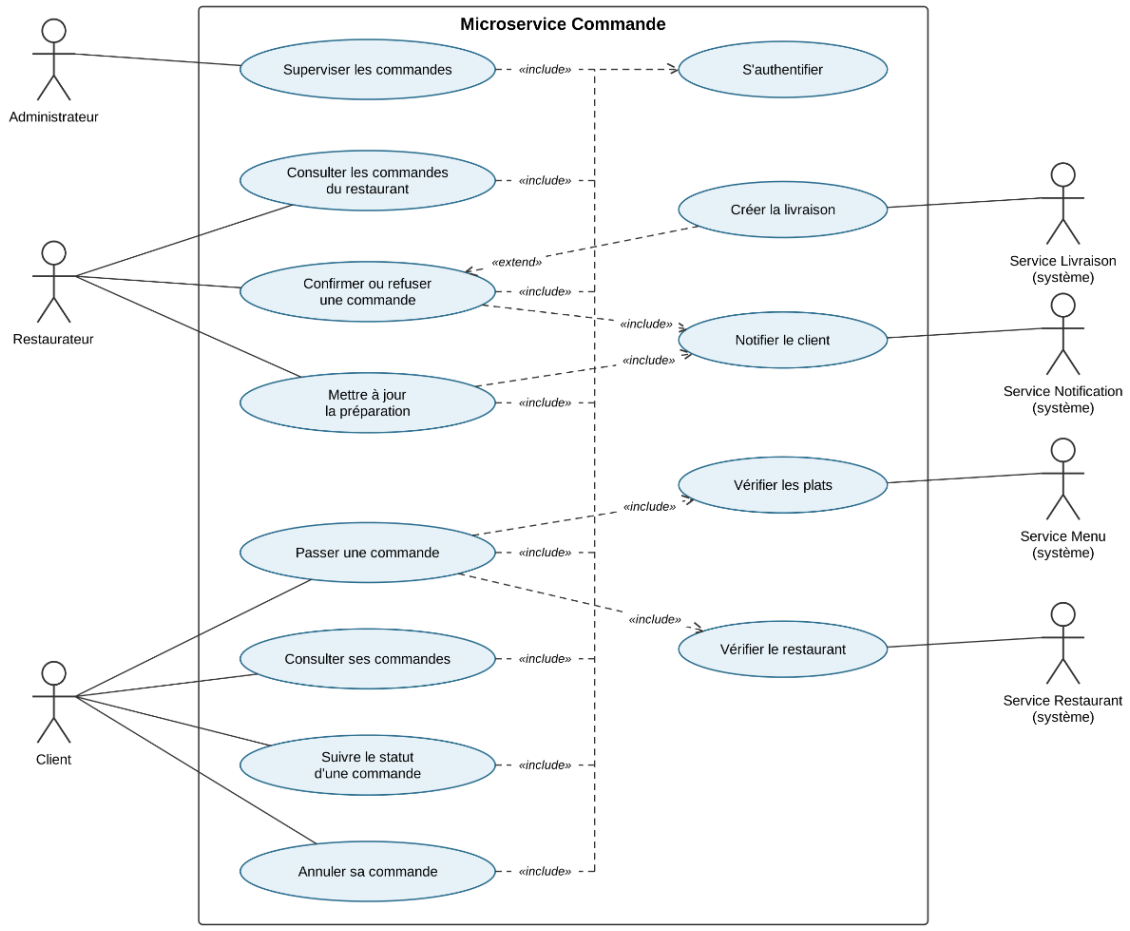


Figure : Digramme de cas d'utilisation Commande

5.1.Fonctionnalités du microservice Commande

Le microservice Commande permet aux clients de gérer leurs commandes auprès des différents restaurants de la plateforme YallaFood.

Ses principales fonctionnalités sont :

1. Consulter le menu du restaurant

- Afficher les plats proposés par le restaurant.
- Consulter les descriptions et les prix des plats.
- Consulter la disponibilité des plats avant de commander.

2. Créer une commande

- Sélectionner un restaurant.
- Choisir les plats et leurs quantités.
- Choisir le mode : à emporter ou livraison.
- Envoyer la commande.

3. Vérifier les plats et le restaurant

- Vérifier le prix et la disponibilité des plats auprès du microservice Menu.
- Vérifier que le restaurant est actif auprès du microservice Restaurant.
- Accepter la commande si tout est valide.
- Refuser la commande si un plat est indisponible ou si le restaurant est inactif.

4. Confirmer une commande

- Enregistrer la commande dans MySQL.
- Attribuer un identifiant unique à la commande.
- Retourner une confirmation au client.
- Permettre au restaurateur de confirmer ou de refuser la commande.

5. Suivre la préparation

- Permettre au restaurateur de faire avancer le statut (EN_PREPARATION, PRETE, LIVREE).
- Mettre à jour le statut selon l'avancement de la livraison.
- Informer le client à chaque changement de statut.

6. Consulter les commandes

- Afficher la liste des commandes du client.
- Consulter les détails d'une commande.
- Vérifier le statut de chaque commande.

7. Annuler une commande

- Permettre au client d'annuler une commande tant qu'elle n'est pas en préparation.
- Mettre à jour son statut.
- Informer les services concernés de l'annulation.

6. Diagramme de classes

Le diagramme de classes présente la structure du microservice Commande ainsi que les relations entre les différentes classes.

Elle représente une réservation effectuée par un client.

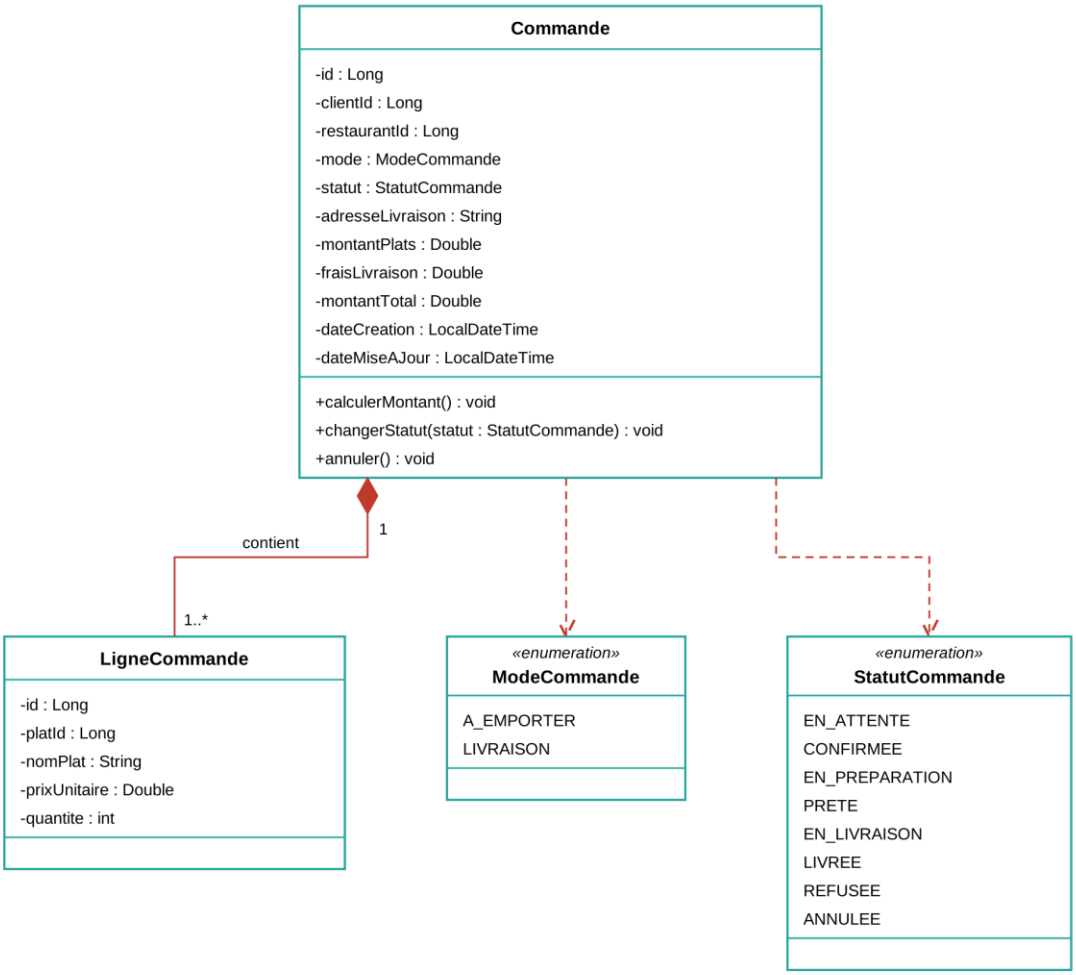


Figure : Digramme de classe Commande

1. Commande

Elle représente une commande passée par un client.

Attributs :

- idCommande
- clientId
- restaurantId
- mode
- statut
- adresseLivraison
- montantPlats
- fraisLivraison
- montantTotal
- dateCreation
- dateMiseAJour

2. LigneCommande

Elle représente un plat appartenant à une commande.

Attributs :

- idLigne
- platId
- nomPlat
- prixUnitaire
- quantite
- sousTotal

3. ModeCommande

Énumération représentant les modes de commande :

- A_EMPORTER
- LIVRAISON

4. StatutCommande

Énumération représentant les différents états d'une commande :

- EN_ATTENTE
- CONFIRMEE
- EN_PREPARATION
- PRETE
- EN_LIVRAISON
- LIVREE
- REFUSEE
- ANNULEE

Relations entre les classes :

- Une commande contient une ou plusieurs lignes de commande.
- Chaque ligne de commande appartient à une seule commande.
- Une commande possède un seul mode de commande.
- Une commande possède un seul statut.

7. Diagramme de séquence

Le diagramme de séquence représente le déroulement chronologique des interactions entre les différents acteurs et composants du microservice Commande lors de la création d'une Commmande .

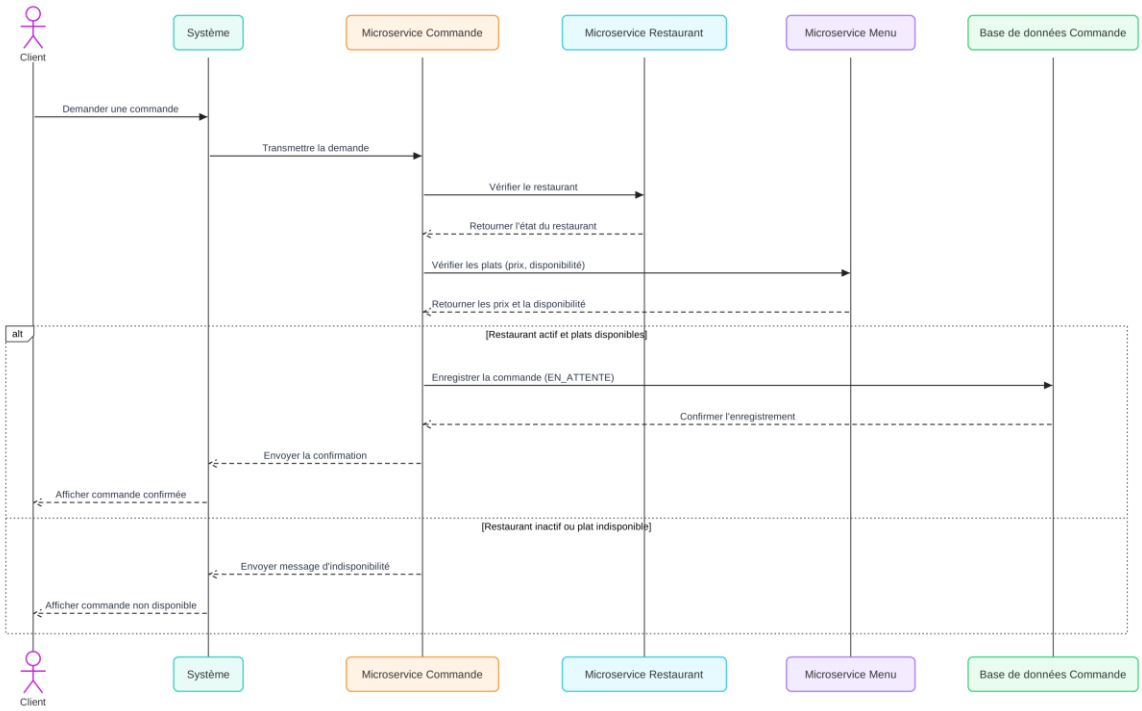


Figure : Digramme de séquence Commande

8. Modèle de données et MySql

Le microservice Commande utilise MySQL comme système de gestion de base de données relationnelle.

Cette base permet de stocker et d'organiser les informations relatives aux commandes.

Structure de la base de données :

Table	Description
commande	Stocke les informations des commandes : client, restaurant, mode, statut, adresse de livraison, montants et dates
ligne_commande	Contient les plats de chaque commande : plat, nom et prix au moment de la commande, quantité

Les avantages de MySQL :

- Garantit la cohérence des données.
- Assure la sécurité des transactions.
- Permet de gérer les relations entre les données (Commande – Lignes de commande).
- Facilite les opérations d'ajout, de modification et de suppression.

Dans notre architecture, les données de commande sont gérées indépendamment par le microservice Commande. Les identifiants du client, du restaurant et des plats sont de simples références vers les autres microservices.

9. API REST et endpoints

L'API REST permet au client et aux autres microservices de communiquer avec le microservice Commande à travers des requêtes HTTP.

Les échanges de données sont effectués au format JSON.

Méthod HTTP	Endpoint	Fonction

POST	/api/commandes	Créer une commande
GET	/api/commandes	Afficher les commandes
GET	/api/commandes/{id}	Consulter une commande
PUT	/api/commandes/{id}/statut	Modifier une Commande
DELETE	/api/commandes/{id}	Annuler une Commande

9.1. Les codes de réponse HTTP

Les codes HTTP permettent d'indiquer le résultat de chaque requête envoyée au microservice Commande .

Code HTTP	Signification	Description
200	OK	Requête exécutée avec succès
201	Created	Données envoyées invalides
400	Bad Request	Réservation introuvable
404	Not Found	Conflit ou indisponibilité de la réservation
409	Conflict	Conflit ou indisponibilité de la réservation
500	Internal Server Error	Erreur interne du serveur

9.3. Exemple d'une requête API

Lorsqu'un client souhaite effectuer une réservation, il envoie une requête POST contenant les informations nécessaires.

```
POST /api/commandes
Content-Type: application/json

{
  "clientId": 25,
  "restaurantId": 7,
  "mode": "LIVRAISON",
  "adresseLivraison": "12 rue de Marseille, Tunis",
  "lignes": [
    { "platId": 12, "quantite": 2 },
    { "platId": 15, "quantite": 1 }
  ]
}
```

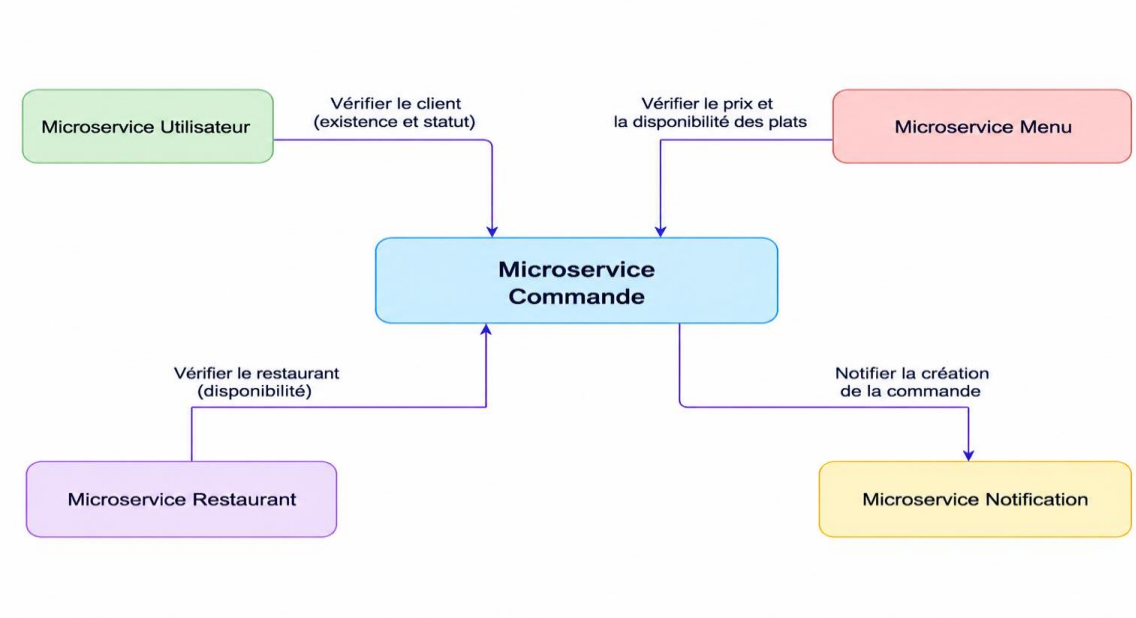
```
{
  "message": "Commande créée avec succès",
  "idCommande": 101,
  "statut": "EN_ATTENTE",
  "montantTotal": 54.0
}
```

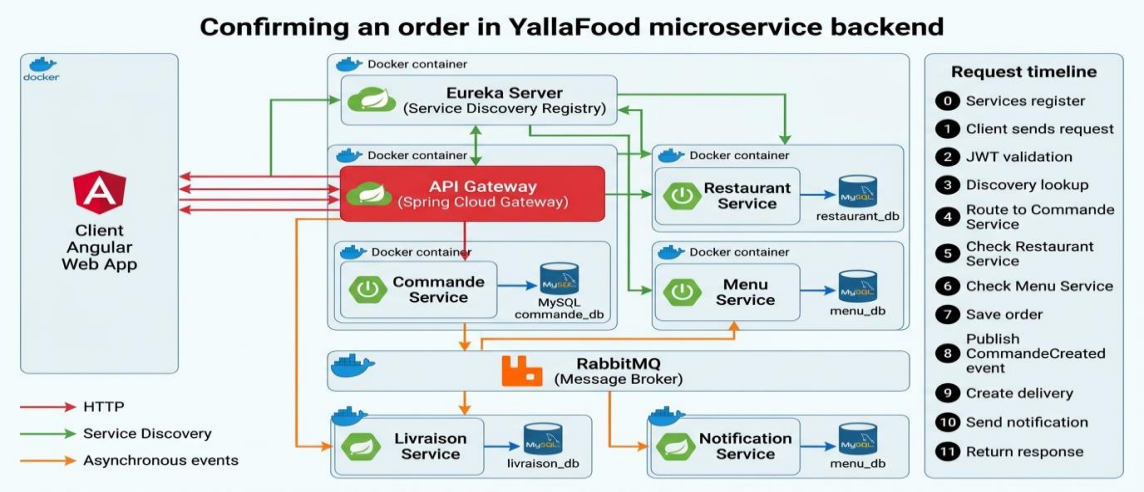
10. Interactions avec les autres microservices

Le microservice Commande ne fonctionne pas seul. Il communique avec les autres microservices pour réaliser le processus de commande.

Interaction avec le microservice Menu :

Le microservice Commande demande au microservice Menu de vérifier le prix et la disponibilité de chaque plat choisi par le client.





Microservice	Pourquoi communiquer ?	Données échangées
Utilisateur	Identifier le client	ID, identité
Restaurant	Vérifier que le restaurant est actif	ID restaurant
Menu	Vérifier le prix et la disponibilité des plats	ID plat, prix, disponibilité
Livraison	Créer la livraison et suivre son avancement	ID commande, adresse, statut
Notification	Informer le client	ID réservation, statut, date

11. Communication synchrone et asynchrone

Dans une architecture microservices, les services peuvent communiquer de deux manières : de façon synchrone ou asynchrone.

A. Communication synchrone

La communication synchrone consiste à envoyer une requête à un service et à attendre directement sa réponse avant de poursuivre le traitement.

Dans notre projet :

- Le microservice Commande envoie une requête HTTP au microservice Restaurant pour vérifier que le restaurant existe et est actif.
- Il envoie une requête HTTP au microservice Menu pour vérifier le prix et la disponibilité des plats.
- Il attend les réponses avant d'enregistrer la commande.

Technologie utilisée : HTTP / API REST.

Exemple : vérifier immédiatement qu'un plat est disponible avant de créer la commande.

B. Communication asynchrone

La communication asynchrone permet aux microservices d'échanger des messages sans attendre immédiatement une réponse.

Dans notre projet, elle est utilisée pour :

- Informer le client et le restaurateur des changements de statut.
- Informer le microservice Livraison qu'une commande en mode livraison est confirmée.
- Recevoir les informations sur l'avancement de la livraison.
- Informer les services concernés lorsqu'une commande est annulée.
- Recevoir un événement signalant la désactivation d'un restaurant.

Technologies possibles : RabbitMQ .

11.1. Les événements du microservice

Les événements permettent d'informer les autres microservices qu'une action importante s'est produite dans le système.

Les principaux événements liés au processus de réservation sont :

Événement	Description
CommandeCreeeEvent	Une nouvelle commande a été créée.
CommandeConfirmeeEvent	Une commande a été confirmée par le restaurateur.
CommandeRefuseeEvent	Une commande a été refusée.
CommandeAnnuleeEvent	Une commande a été annulée.
CommandePreteEvent	Une commande est prête à être remise au client ou au livreur.

Ces événements peuvent être transmis de manière asynchrone à l'aide d'un système de messagerie comme RabbitMQ .

12. Conclusion

Le microservice Commande constitue un élément essentiel de notre architecture microservices. Il permet de gérer efficacement le processus de commande de repas, à emporter ou en livraison.

Grâce à l'utilisation de MySQL, des API REST et de la communication entre microservices, il garantit une gestion structurée et indépendante des commandes.

Cette architecture facilite la maintenance, l'évolution et l'intégration de nouvelles fonctionnalités dans le système.

Microservice Livraison

1. Présentation du microservice Livraison

Le microservice Livraison prend en charge la commande une fois celle-ci confirmée en mode « livraison » : il crée la livraison, assigne un livreur, suit l'avancement jusqu'à la remise au client et informe les autres services de chaque changement d'état.

1.1 Responsabilités

- Créer une livraison à partir d'une commande confirmée en mode livraison
- Gérer les livreurs (profil, disponibilité) et leur assigner les livraisons
- Gérer le cycle de vie d'une livraison et son historique de suivi
- Fournir au client le statut, la position et l'heure estimée d'arrivée

- Publier des événements pour synchroniser Commande et Notification

Données gérées : livraisons, livreurs, historique de suivi. Le service ne stocke que des identifiants (commandeId, restaurantId, clientId) pour les données qui appartiennent aux autres services.

2.Acteurs et fonctionnalités

1.2 Acteurs

Acteur	Type	Rôle vis-à-vis du microservice Livraison
Client	Principal	Consulte le statut, le suivi et l'historique de ses livraisons
Livreur	Principal	Consulte ses livraisons assignées, met à jour leur statut, envoie sa position et gère sa disponibilité
Restaurateur	Principal	Suit les livraisons de son restaurant, assigne un livreur et peut annuler une livraison non récupérée
Service Notification	Secondaire (système)	Prévient le client à chaque changement de statut
Administrateur	Principal	Ajouter, modifier, supprimer et consulter les livreurs

1.3 Fonctionnalités

N°	Fonctionnalité	Acteur	Description
F1	S'authentifier	Client, Livreur, Restaurateur, Administrateur	Se connecter à la plateforme (jeton JWT) avant d'accéder à toute fonctionnalité
F2	Consulter le statut et le suivi d'une livraison	Client	Voir le statut, la position du livreur et l'heure estimée d'arrivée
F3	Consulter ses livraisons	Client	Voir l'historique de ses livraisons
F4	Annuler sa livraison	Client	Annuler sa livraison tant que le repas n'est pas récupéré
F5	Consulter ses livraisons assignées	Livreur	Voir la liste des livraisons à effectuer

F6	Mettre à jour le statut d'une livraison	Livreur	Faire avancer la livraison : RECUPEREE, EN_ROUTE, LIVREE ou ECHOUEE
F7	Envoyer sa position	Livreur	Enregistrer sa position pour alimenter le suivi
F8	Changer sa disponibilité	Livreur	Passer DISPONIBLE, OCCUPE ou HORS_LIGNE
F9	Suivre les livraisons du restaurant	Restaurateur	Voir l'état des livraisons liées à son restaurant
F10	Assigner un livreur	Restaurateur	Associer un livreur DISPONIBLE à une livraison
F11	Gérer les livreurs	Administrateur	Ajouter, modifier, supprimer et consulter les livreurs
F12	Notifier le client	Service Notification	Envoi d'une notification à chaque changement de statut (inclus dans F6)

1.4 Technologies

Élément	Choix et justification
Framework	Spring Boot (Spring Web, Spring Data JPA, validation)
Base de données	MySQL, base propre au service (livraison_db) : données structurées, relations Livreur–Livraison–Suivi, transactions pour l'assignation d'un livreur
Communication synchrone	OpenFeign via Eureka / API Gateway
Communication asynchrone	RabbitMQ
Sécurité	Jeton JWT vérifié au niveau de l'API Gateway, contrôle des rôles dans le service
Documentation d'API	Swagger / OpenAPI

3. Diagramme de cas d'utilisation

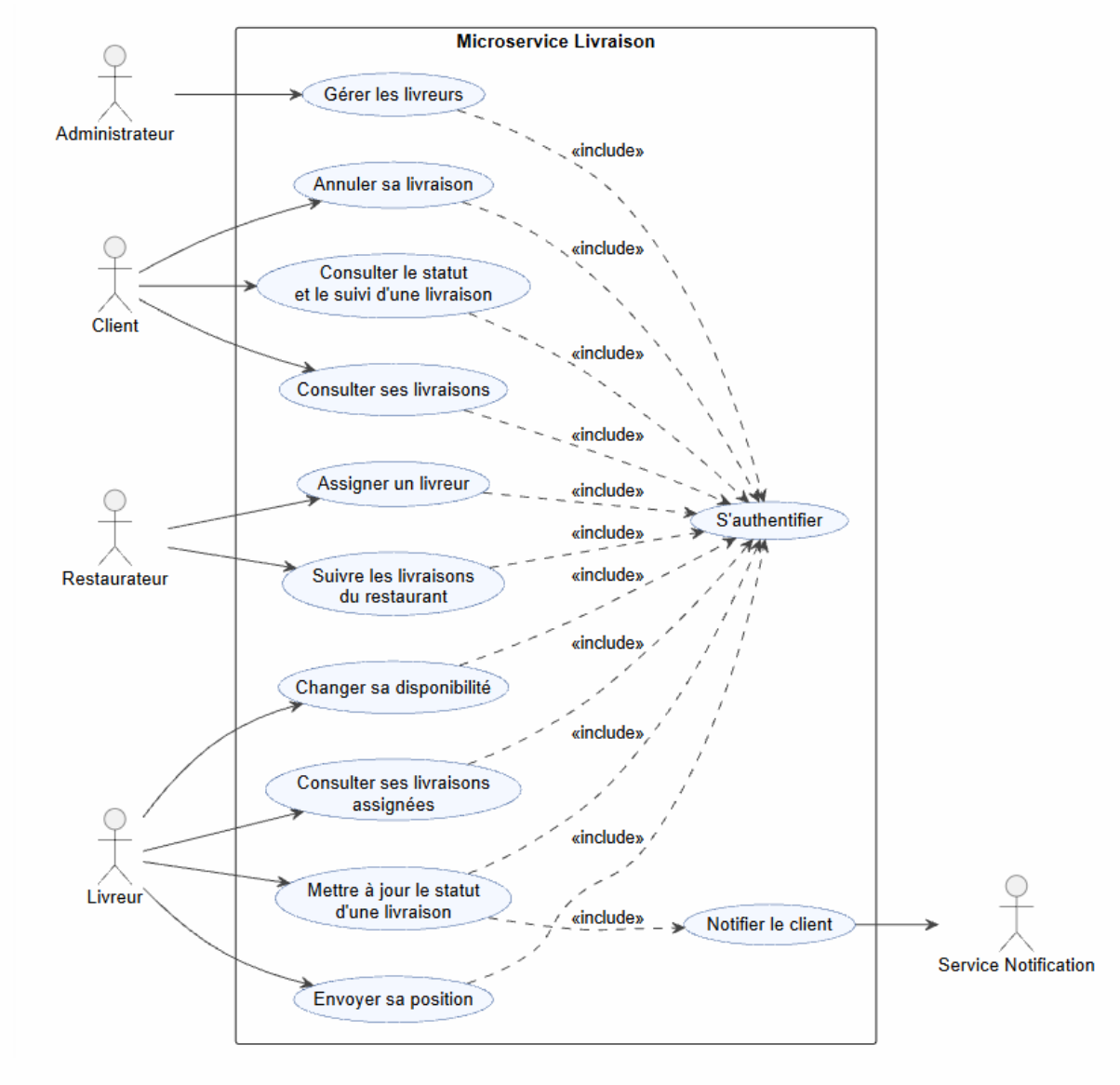


Figure 1 – Diagramme de cas d'utilisation du microservice Livraison

Les acteurs principaux (Client, Livreur, Restaurateur, Administrateur) interagissent avec le service via l'API Gateway. Le Service Commande déclenche la création d'une livraison, et le Service Notification est sollicité à chaque changement de statut (« include » de la mise à jour du statut). La création d'une livraison inclut l'assignation d'un livreur.

4. Diagramme de classes

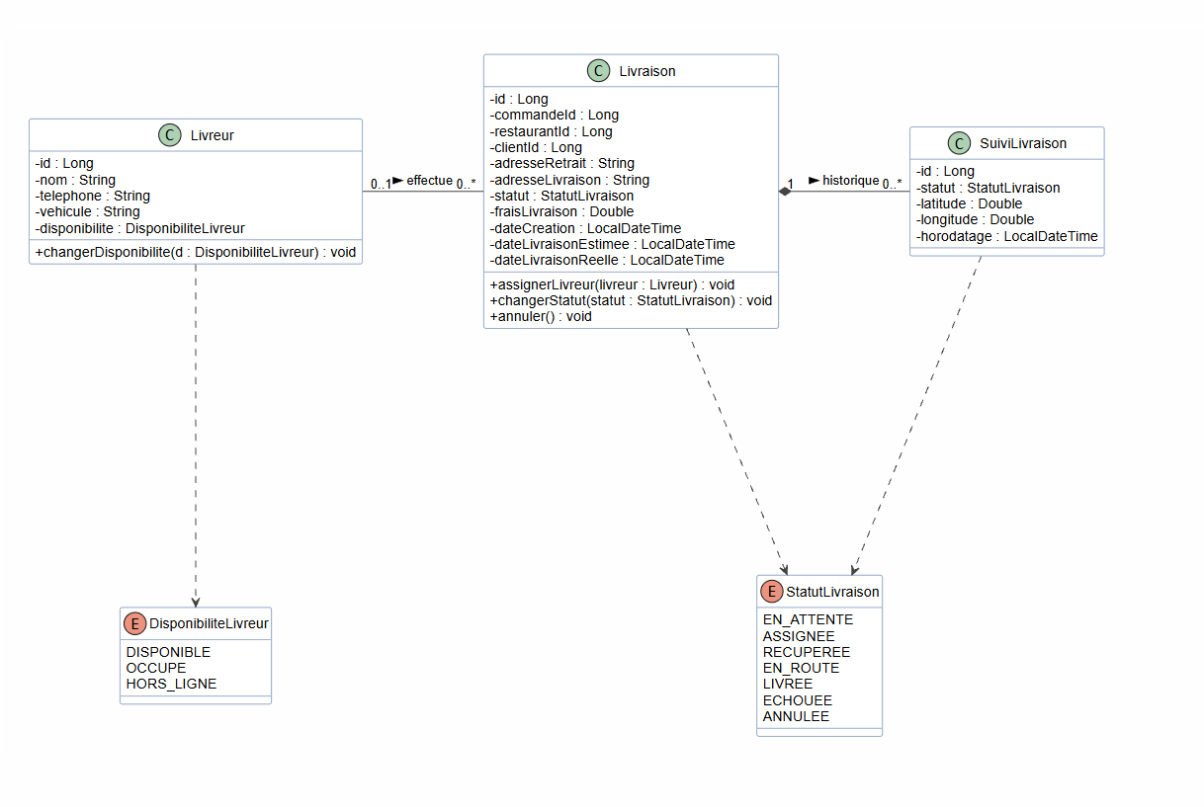


Figure 2 – Diagramme de classes du microservice Livraison

4.1 Cycle de vie d'une livraison

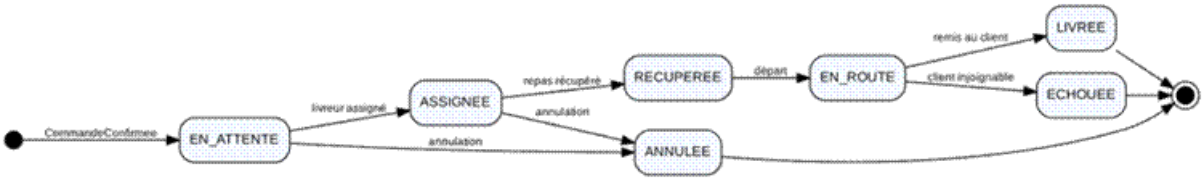


Figure 3 – Diagramme d'états d'une livraison

5. Diagramme de séquence

Scénario principal : de la confirmation de la commande jusqu'à la livraison terminée.

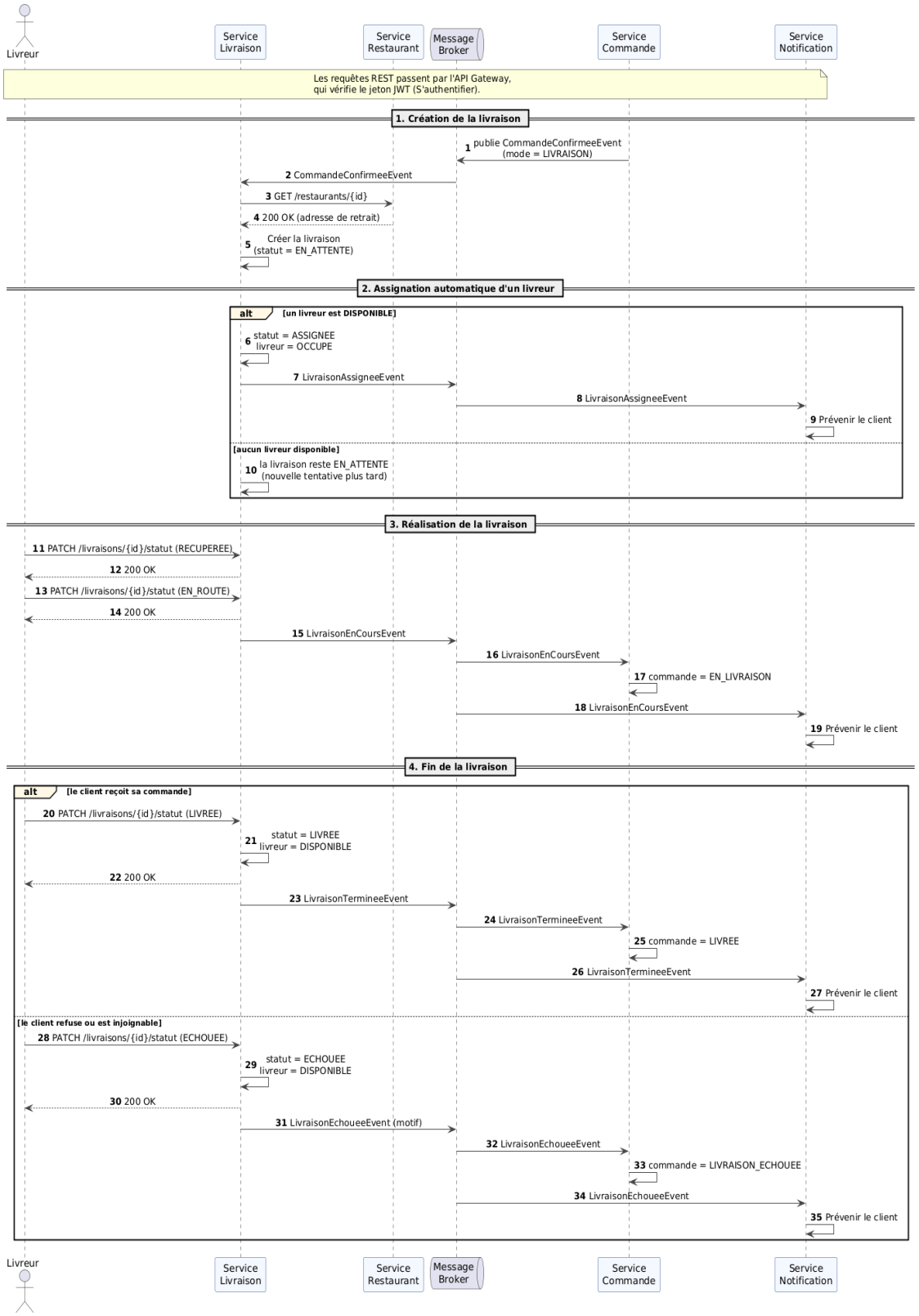


Figure 4 – Diagramme de séquence : de la commande confirmée à la livraison terminée

6. Modèle de données et MySQL

Le microservice Livraison utilise MySQL comme système de gestion de base de données relationnelle.

Cette base permet de stocker et d'organiser les informations relatives aux livraisons, aux livreurs et au suivi des commandes.

Structure de la base de données :

Table	Contenu
livreur	Livreurs de la plateforme : nom, téléphone, véhicule et disponibilité (DISPONIBLE, OCCUPE, HORS_LIGNE)
livraison	Une ligne par livraison : références de la commande, du restaurant et du client, livreur assigné (nullable), adresses, statut, frais et dates. commande_id est unique : une commande ne génère qu'une livraison
suivi_livraison	Historique des changements de statut et des positions GPS d'une livraison (supprimé en cascade avec la livraison)

Figure 5 – Modèle de données du microservice Livraison

Les avantages de MySQL :

- Garantit la cohérence des données.
- Assure la sécurité des transactions (par exemple lors de l'assignation d'un livreur).
- Permet de gérer les relations entre les données (Livreur – Livraison – Suivi).
- Facilite les opérations d'ajout, de modification et de suppression.

Dans notre architecture, les données de livraison sont gérées indépendamment par le microservice Livraison. Les identifiants de commande, de restaurant et de client sont de simples références vers les autres microservices.

7. API REST et endpoints

Toutes les requêtes passent par l'API Gateway (préfixe /api). Format d'échange : JSON. Accès sécurisé par jeton JWT : le rôle LIVREUR met à jour les statuts, le rôle ADMIN ou RESTAURATEUR gère les assignations, le rôle CLIENT consulte le suivi de ses propres livraisons.

7.1 Livraisons

Méthode	Endpoint	Description	Codes de réponse

POST	/api/livraisons	Créer une livraison (manuel ou via événement)	201 Created, 400 Bad Request, 404 Commande introuvable, 409 Livraison déjà existante
GET	/api/livraisons/{id}	Détails d'une livraison	200 OK, 404 Not Found
GET	/api/livraisons?clientId=&statut=	Lister les livraisons (filtres)	200 OK
PATCH	/api/livraisons/{id}/statut	Changer le statut	200 OK, 400, 404, 409 Transition interdite
PUT	/api/livraisons/{id}/livreur	Assigner un livreur	200 OK, 404, 409 Livreur indisponible
GET	/api/livraisons/{id}/suivi	Historique et dernière position	200 OK, 404
POST	/api/livraisons/{id}/suivi	Enregistrer une position	201 Created, 404
DELETE	/api/livraisons/{id}	Annuler une livraison (si pas encore RECUPEREE)	204 No Content, 404, 409

7.2 Livreurs

Méthode	Endpoint	Description	Codes de réponse
POST	/api/livreurs	Ajouter un livreur	201 Created, 400
GET	/api/livreurs?disponible=true	Lister les livreurs (filtre disponibilité)	200 OK
GET	/api/livreurs/{id}	Détails d'un livreur	200 OK, 404
PUT	/api/livreurs/{id}	Modifier un livreur	200 OK, 400, 404
PATCH	/api/livreurs/{id}/disponibilite	Changer la disponibilité	200 OK, 404

7.3 Exemples

Requête POST /api/livraisons :

```
{
  "commandeId": 101,
  "restaurantId": 7,
  "clientId": 25,
  "adresseLivraison": "12 rue de Marseille, Tunis"
}
```

Réponse 201 Created :

```
{
  "id": 55,
  "statut": "EN_ATTENTE",
  "fraisLivraison": 5.0,
  "dateLivraisonEstimee": "2026-11-12T13:15:00"
}
```

Requête PATCH /api/livraisons/55/statut :

```
{ "statut": "EN_ROUTE" }
```

8. Interactions avec les autres microservices

Le microservice Livraison ne fonctionne pas seul. Il communique avec les autres microservices pour réaliser le processus de livraison d'une commande.

Interaction avec le microservice Commande :

Lorsqu'une commande est confirmée en mode livraison, le microservice Commande publie l'événement CommandeConfirmeeEvent. Le microservice Livraison le reçoit et crée la livraison, puis informe Commande de son avancement (assignée, en cours, terminée, échouée).

Interaction avec le microservice Restaurant :

Le microservice Livraison demande au microservice Restaurant l'adresse du restaurant afin de définir le point de retrait du repas par le livreur.

Interaction avec le microservice Notification :

À chaque changement important du statut de la livraison, le microservice Livraison publie un événement qui permet au microservice Notification de prévenir le client.

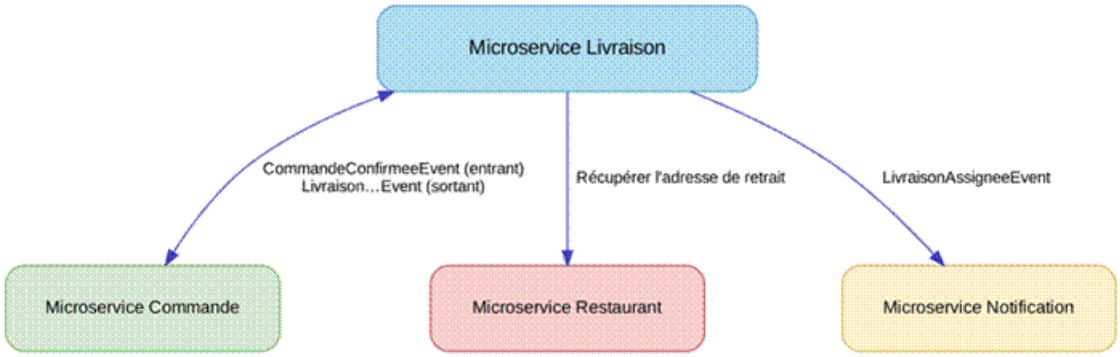


Figure 6 – Interactions du microservice Livraison avec les autres microservices

9.Communications synchrones et asynchrones

Type	Interaction	Objectif

Synchrone (REST)	Livraison → Restaurant : GET /restaurants/{id}	Récupérer l'adresse de retrait du restaurant
Synchrone (REST)	Livraison → Commande : GET /commandes/{id}	Vérifier la commande et récupérer l'adresse du client
Asynchrone (événement)	Commande → Livraison : CommandeConfirmeeEvent	Déclencher la création de la livraison (mode LIVRAISON uniquement)
Asynchrone (événement)	Livraison → Commande : LivraisonAssigneeEvent, LivraisonEnCoursEvent, LivraisonTermineeEvent, LivraisonEchoueeEvent	Mettre à jour le statut de la commande
Asynchrone (événement)	Livraison → Notification	Prévenir le client à chaque étape

10.Événements

Événement	Sens	Producteur → Consommateur	Contenu principal
CommandeConfirmeeEvent	Entrant	Commande → Livraison	commandeId, restaurantId, clientId, adresseLivraison, mode
LivraisonAssigneeEvent	Sortant	Livraison → Commande, Notification	livraisonId, commandeId, livreurId
LivraisonEnCoursEvent	Sortant	Livraison → Commande, Notification	livraisonId, commandeId, dateLivraisonEstimee
LivraisonTermineeEvent	Sortant	Livraison → Commande, Notification	livraisonId, commandeId, dateLivraisonReelle
LivraisonEchoueeEvent	Sortant	Livraison → Commande, Notification	livraisonId, commandeId, motif

11.Conclusion

Le microservice Livraison reste autonome : il utilise le synchrone (REST) pour les informations immédiates et l'asynchrone (événements) pour échanger l'avancement des livraisons avec Commande et Notification, ce qui limite le couplage entre les services.